SOMAIYA VIDYAVIHAR UNIVERSITY

Internal Assessment Report:

"Comprehensive Web Application Vulnerability Assessment and Exploitation using OWASP Juice Shop and Burp Suite"

by

Name: Angel Mariam Shibu
Roll No: 16014024002

Name: Chaitanyaa Dave
Roll No: 16014024014

Name: Aishwarya Lakshmi
Roll No: 16015024038

For the subject

Cyber Forensics & Laws

Department of Electronics & Computer Engineering
Academic Year 2026-27

1. Abstract

This report presents a hands-on security assessment of OWASP Juice Shop, an intentionally vulnerable web application, carried out on a local instance using Burp Suite. After exploring the main functions of the application (browsing, search, registration, login and cart), five tests were performed: authentication and session management, broken access control, input validation, security misconfiguration, and cart and business logic.

Login and logout behaved as expected, and a basic comparison of two accounts did not show unauthorized access. Unexpected search input caused an HTTP 500 response that disclosed a SQLite error and the SQL query, and the cart accepted a negative quantity. The report records the procedure, evidence and conclusion for each test, together with recommendations to remedy the issues found.

Keywords: web application security, OWASP Juice Shop, Burp Suite, authentication, input validation, business logic.

2. Introduction

This report documents a hands-on web application security assessment of OWASP Juice Shop, a deliberately vulnerable web application built for security training. The application was run locally at http://localhost:3000 and all browser traffic was routed through Burp Suite so that HTTP requests and responses could be captured, inspected and replayed.

Tools used

- OWASP Juice Shop – the target web application, running on localhost:3000.
- Burp Suite – Proxy (Intercept and HTTP history) to capture traffic, and Repeater to modify and resend requests.
- Burp Suite's built-in browser – used to access the application through the proxy.

3. Objectives

The objective of this work was to understand how a typical web application behaves under security testing and to practise recording evidence responsibly. The specific objectives were:

- To set up OWASP Juice Shop locally and route its traffic through Burp Suite.

- To explore the main functions of the application and study their requests and responses.
- To test authentication and session management, including login and logout behaviour.
- To check for broken access control between two test accounts.
- To test input validation using empty, long and unexpected input.
- To review response headers and error messages for security misconfiguration.
- To examine the cart for business logic weaknesses such as invalid quantities.
- To record the evidence for each test, judge its real security impact and suggest remedies.

All testing was performed on a local instance using test accounts and test data only.

4. Features and Areas of Focus of the Platform and Tools

4.1 OWASP Juice Shop

OWASP Juice Shop is an intentionally insecure web application maintained by the OWASP community for security training, demonstrations and awareness. It is built with Node.js, an Express back end, an Angular front end and an SQLite database, and it imitates an online juice shop with a product catalogue, search, user registration and login, a shopping basket, user profiles and feedback features. It deliberately contains vulnerabilities from across the OWASP Top 10, so it can be attacked safely, but it must only be run locally and never exposed to the internet.

4.2 Burp Suite

Burp Suite is a web security testing tool from PortSwigger that sits between the browser and the application as an intercepting proxy. The features used in this work were:

- Proxy: passes all browser traffic through Burp so that every request and response can be observed.
- Intercept: holds a request before it reaches the server so that it can be inspected or changed.
- HTTP history: keeps a log of all requests and responses, which served as the evidence record for this report.
- Repeater: lets a captured request be modified and sent again, and the responses compared.
- Built-in browser: a browser that is already set up to use Burp's proxy.

4.3 Areas of focus and OWASP Top 10 mapping

The five tests were chosen to cover common web application weaknesses. Each is related to a category of the OWASP Top 10 (2025 edition), as shown below.

Test	Area of focus	Related OWASP Top 10 (2025) category
1	Authentication and session management	A07: Authentication Failures
2	Broken access control	A01: Broken Access Control
3	Input validation	A05: Injection (disclosure also relates to A10: Mishandling of Exceptional Conditions)
4	Security misconfiguration	A02: Security Misconfiguration
5	Cart and business logic	A06: Insecure Design

5. Environment Setup

5.1 Install Node.js

1. Open the official website: Node.js downloads.

2. Download the Windows x64 LTS installer if your PC has an Intel or AMD 64-bit processor.

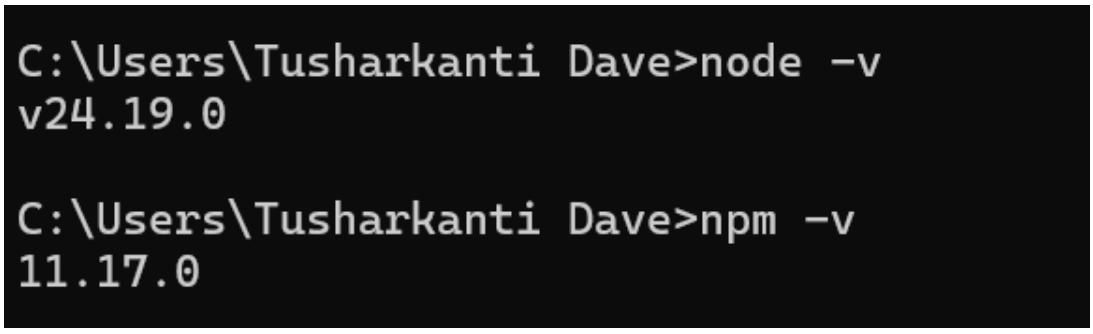
3. Run the installer and keep the default options.

4. Open Command Prompt and check the installation:

node -v

npm -vs

Both commands should display version numbers.



5.2 Download OWASP Juice Shop

1. Open the official releases page: https://github.com/juice-shop/juice-shop/releases/latest

2. Scroll to Assets.

3. Download the ZIP file whose name resembles:

juice-shop-<version>_node24_windows_x64.zip

Choose the file matching your installed Node.js major version. The current project supports Node.js 22 and 24; packaged downloads are available for Windows x64 with Node.js 24.

GitHub

4. Extract the ZIP to a simple folder, such as C:\juice-shop.

5.3 Start Juice Shop

1. Open the extracted folder in File Explorer.

2. Click the address bar, type cmd, and press Enter.

3. Run:

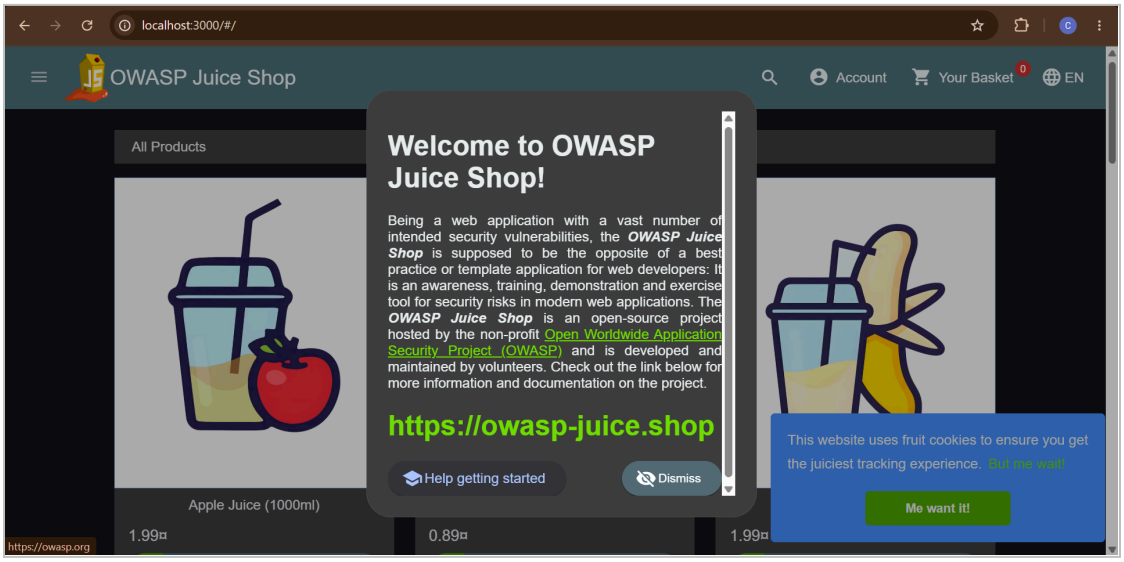
npm start

OWASP Juice Shop was started locally and is served at localhost:3000.

```
C:\CyberSecIA\juice-shop_20.3.0>npm start

> juice-shop@20.3.0 start
> node build/app

info: Detected Node.js version v24.19.0 (SUCCESS)
info: Detected OS win32 (SUCCESS)
info: Detected CPU x64 (SUCCESS)
info: Configuration default validated (SUCCESS)
info: Entity models 21 of 21 are initialized (SUCCESS)
info: All dependencies in ./package.json are satisfied (SUCCESS)
info: Required file server.js is present (SUCCESS)
info: Required file styles.css is present (SUCCESS)
info: Required file main.js is present (SUCCESS)
info: Required file index.html is present (SUCCESS)
info: Required file polyfills.js is present (SUCCESS)
info: Required file matching /^hacking-instructor-.+\.js$/ is present (SUCCESS)
info: Port 3000 is available (SUCCESS)
info: Domain https://www.alchemy.com/ is reachable (SUCCESS)
warn: Environment variable ALCHEMY_API_KEY is not present (WARNING)
warn: "Mint the Honey Pot" challenge will not work as intended without a valid ALCHEMY_API_KEY
warn: "Wallet Depletion" challenge will not work as intended without a valid ALCHEMY_API_KEY
info: Check https://howto-web3.owasp-juice.shop for instructions on how to set up and configure the Alchemy API
warn: Domain http://localhost:11434/v1 is not reachable (WARNING)
warn: "Chatbot Prompt Injection" challenge will not work as intended without access to http://localhost:11434/v1
warn: "Greedy Chatbot Manipulation" challenge will not work as intended without access to http://localhost:11434/v1
warn: "AI Debugging" challenge will not work as intended without access to http://localhost:11434/v1
warn: "System Prompt Extraction" challenge will not work as intended without access to http://localhost:11434/v1
info: Check https://howto-llm.owasp-juice.shop for instructions on how to set up and configure the LLM API
warn: File ./infrastructure could not be read. It might have been moved or deleted. If coding challenges are contained in the file, they will not be availab
le.
info: Server listening on port 3000
```



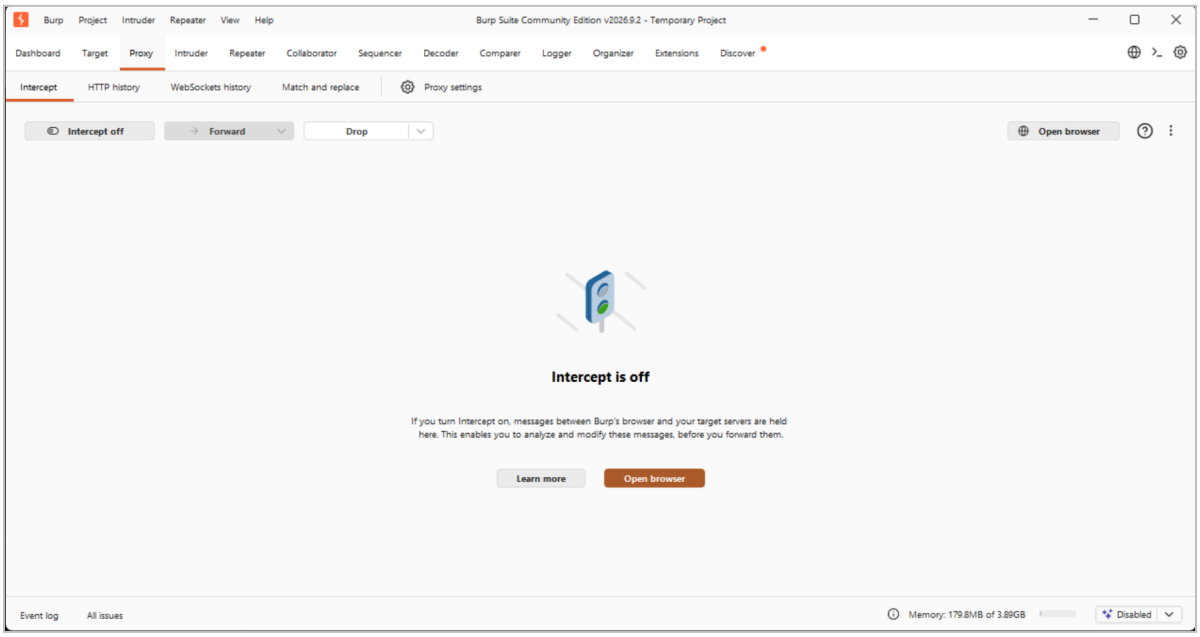
5.4 Configuring Burp Suite

Open Burp Suite:

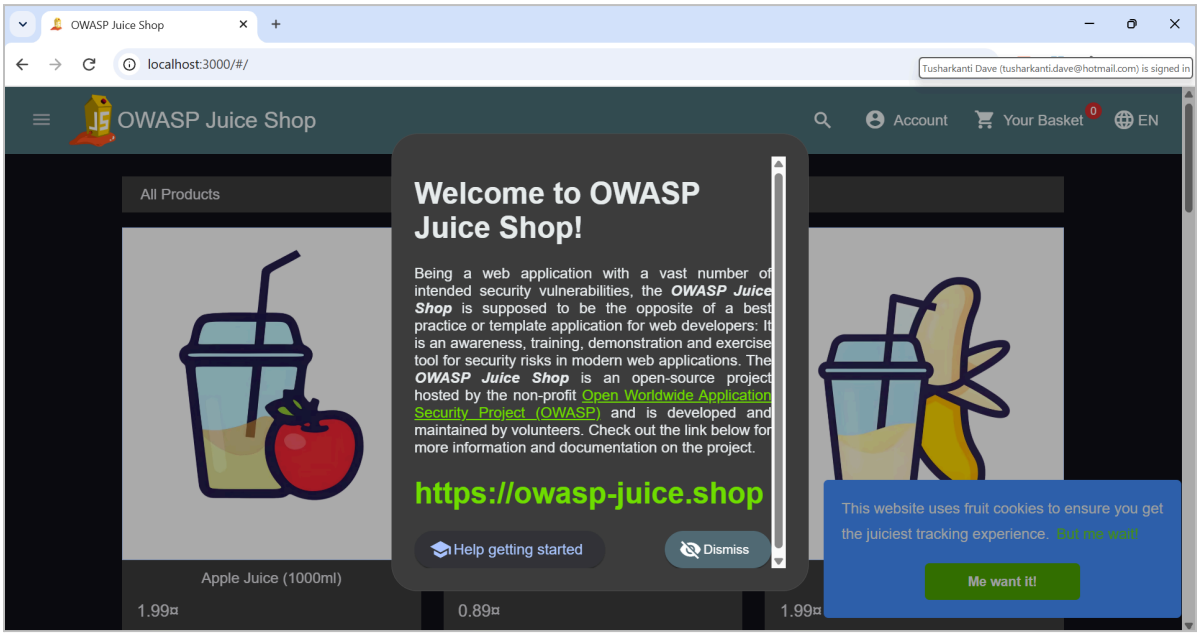
- Launch Burp Suite.
- Select Temporary project if prompted.
- Select Use Burp defaults.
- Click Start Burp.

You should reach the main Burp dashboard.

In Burp Suite, open Proxy tab, then Intercept, and the built-in browser is launched by clicking on Open Browser.



The Juice Shop address, http://localhost:3000, was then opened in this browser.

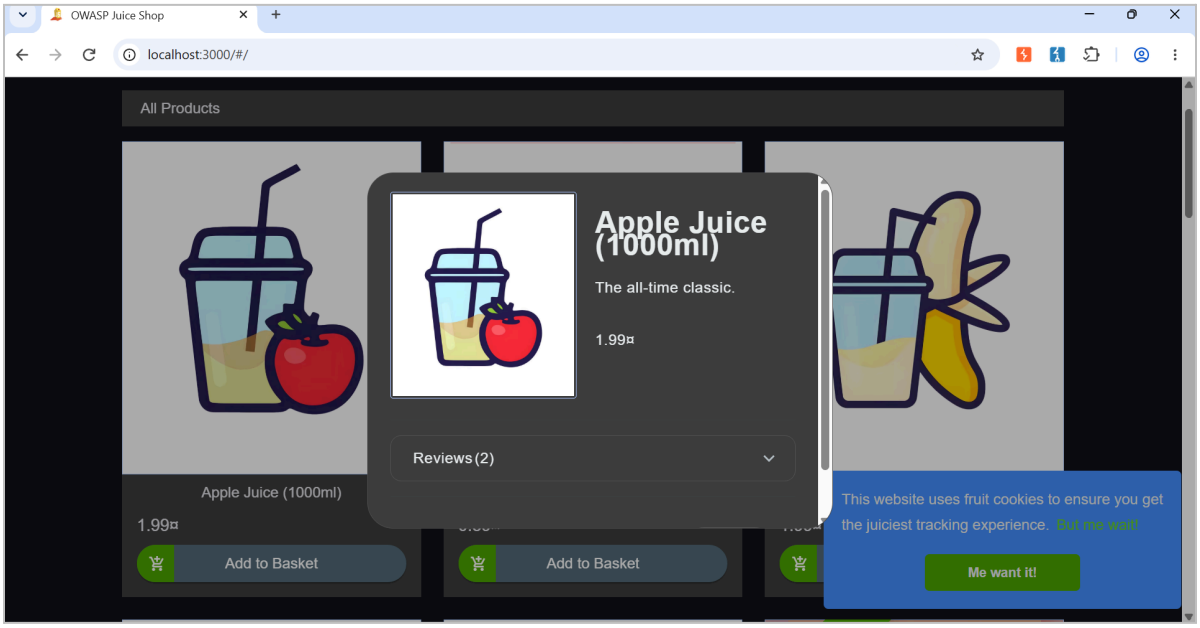


6. Exploring Application Functionality

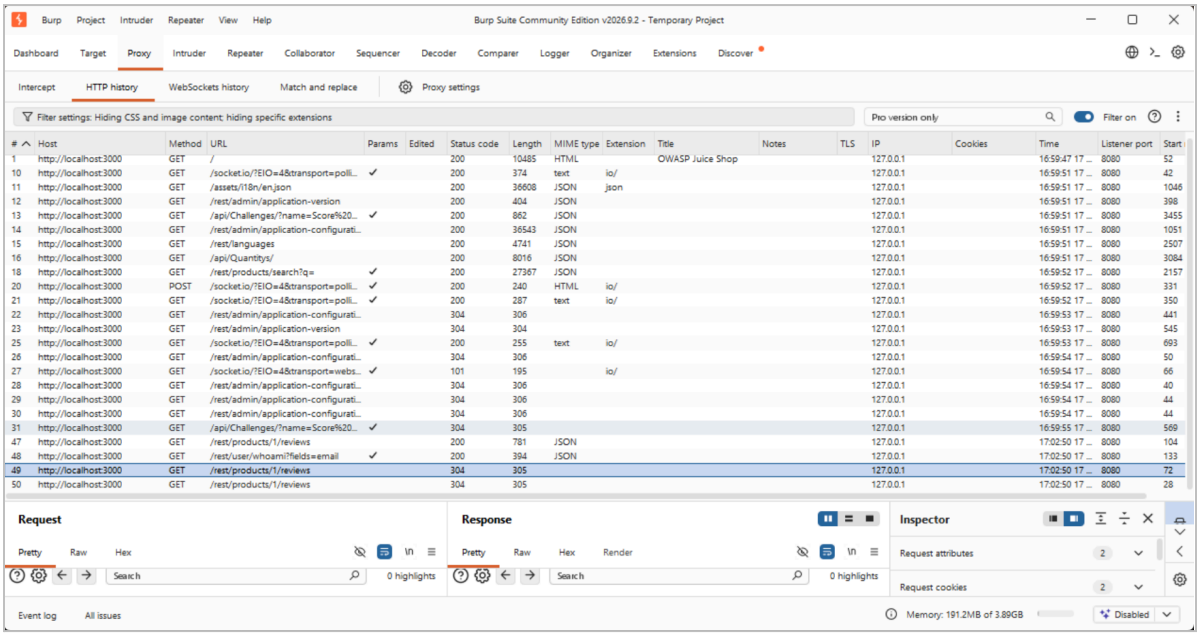
Before testing, the main functions of the application were used normally so that the corresponding requests and responses could be studied in Burp Suite.

6.1 Browsing and searching for products

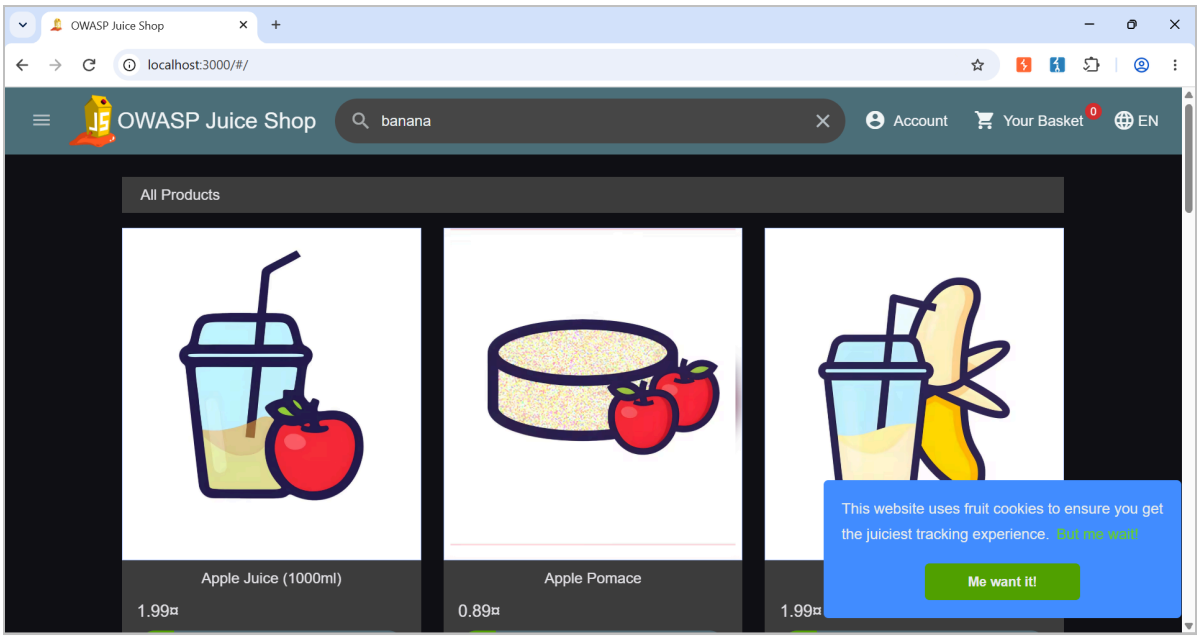
A product was opened from the product listing.

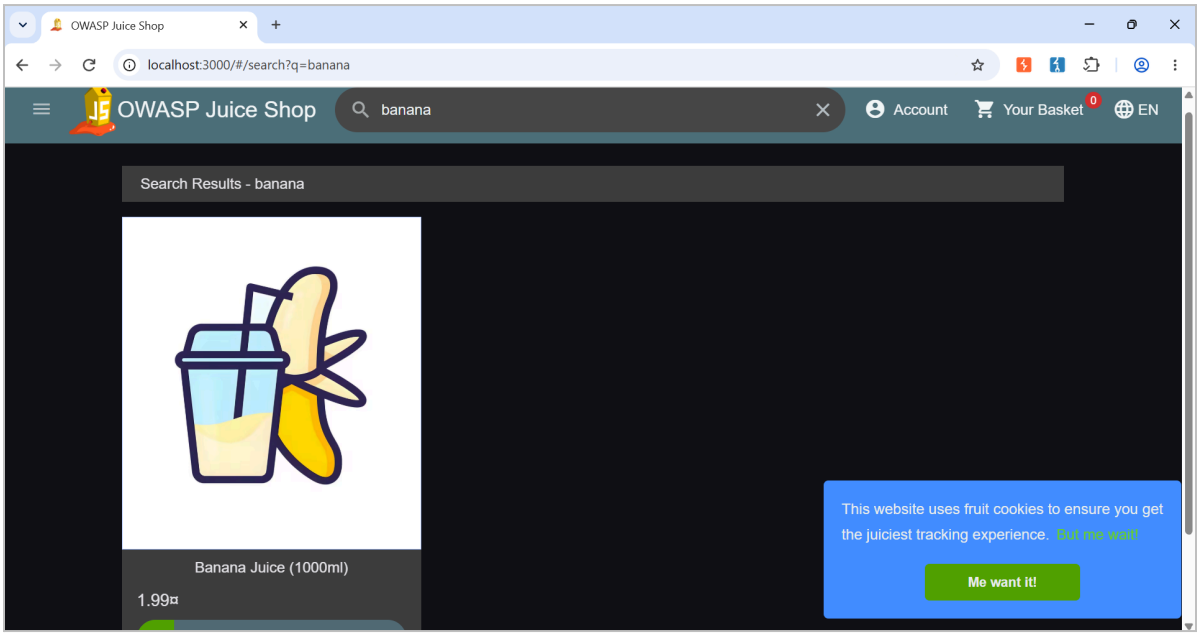


As seen in Burp Suite, the requests are sent to localhost:3000.

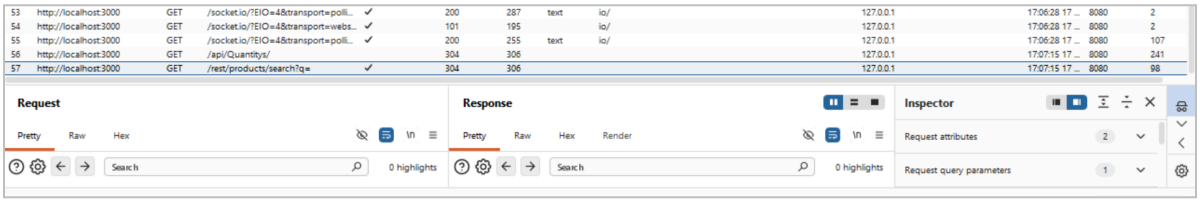


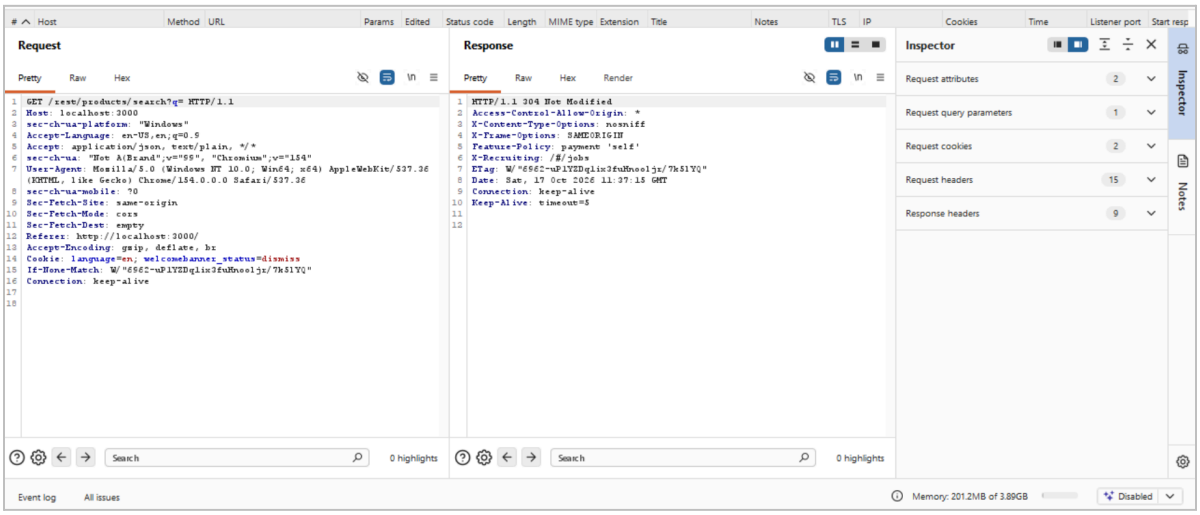
A product search was then performed.



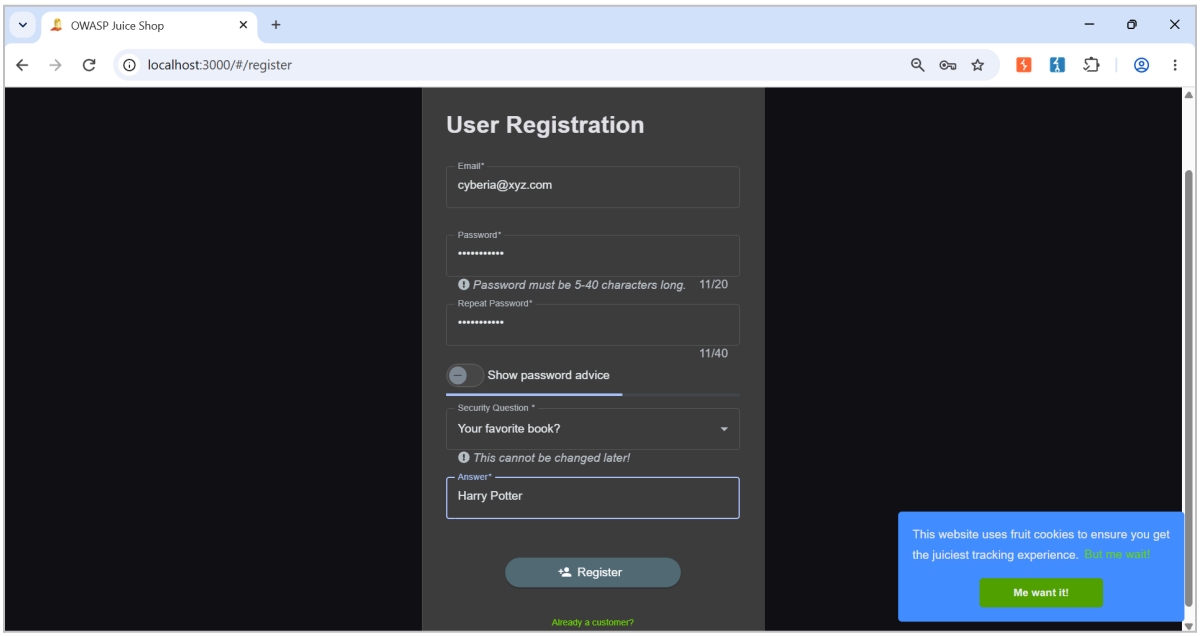


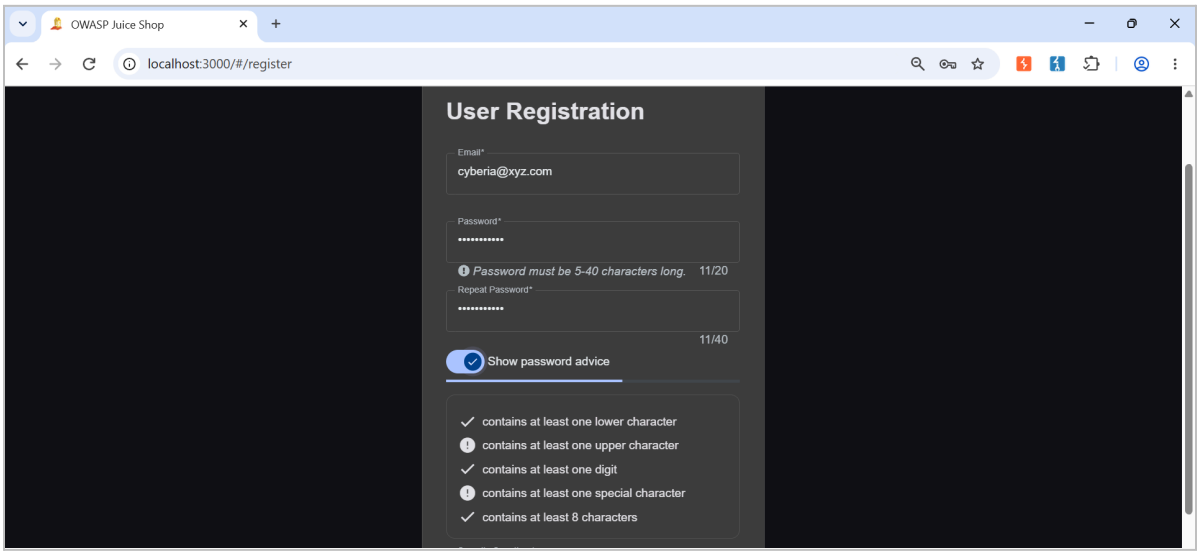
The search request is visible in Burp Suite.



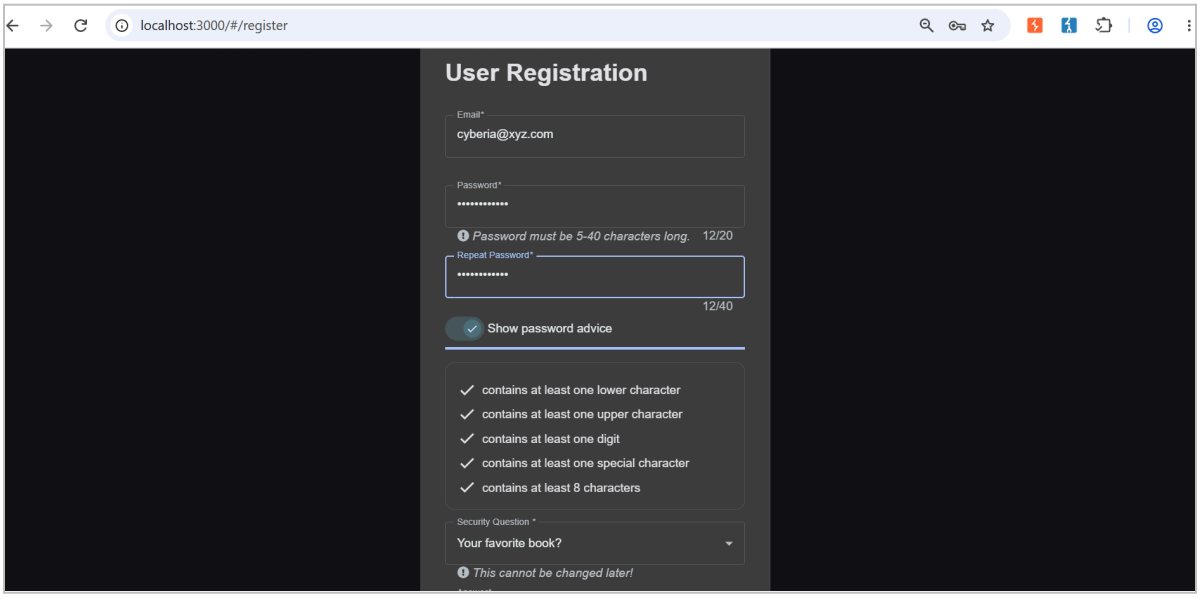


6.2 Registration request and response

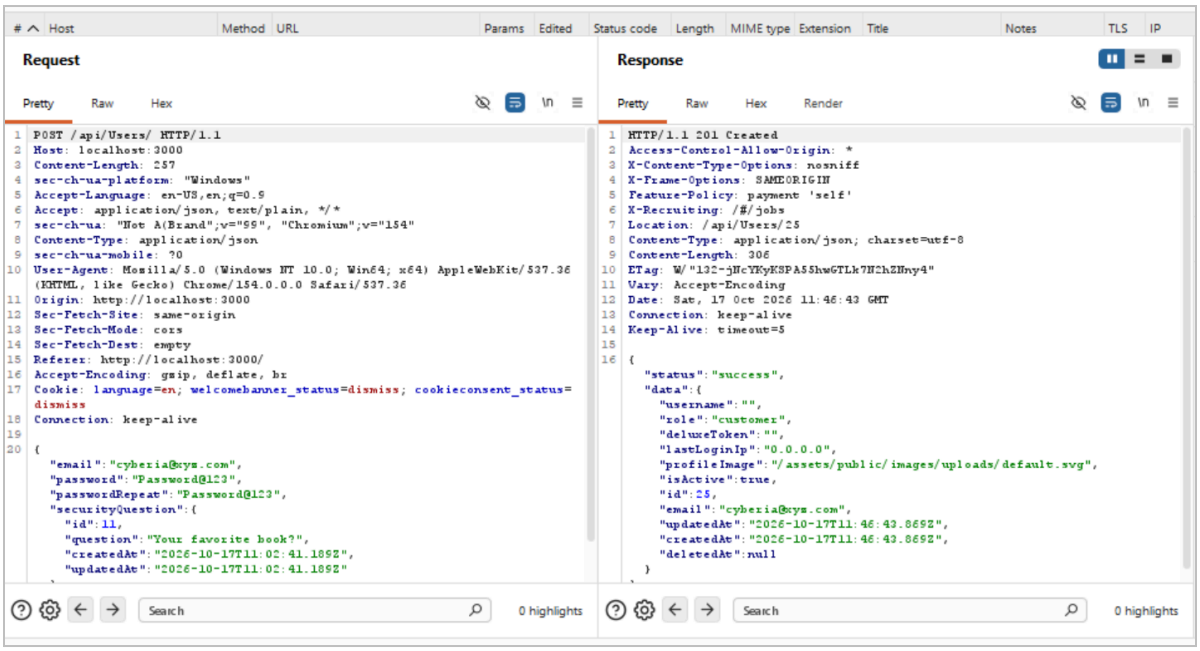


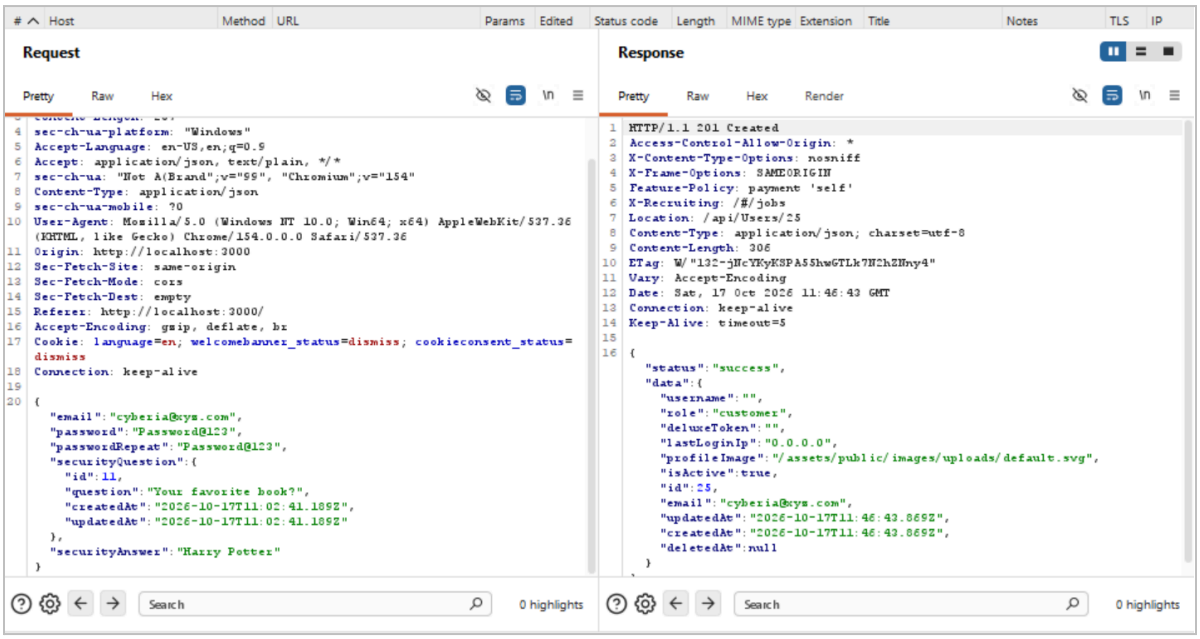


Test account password used: Password@123 (test data only).

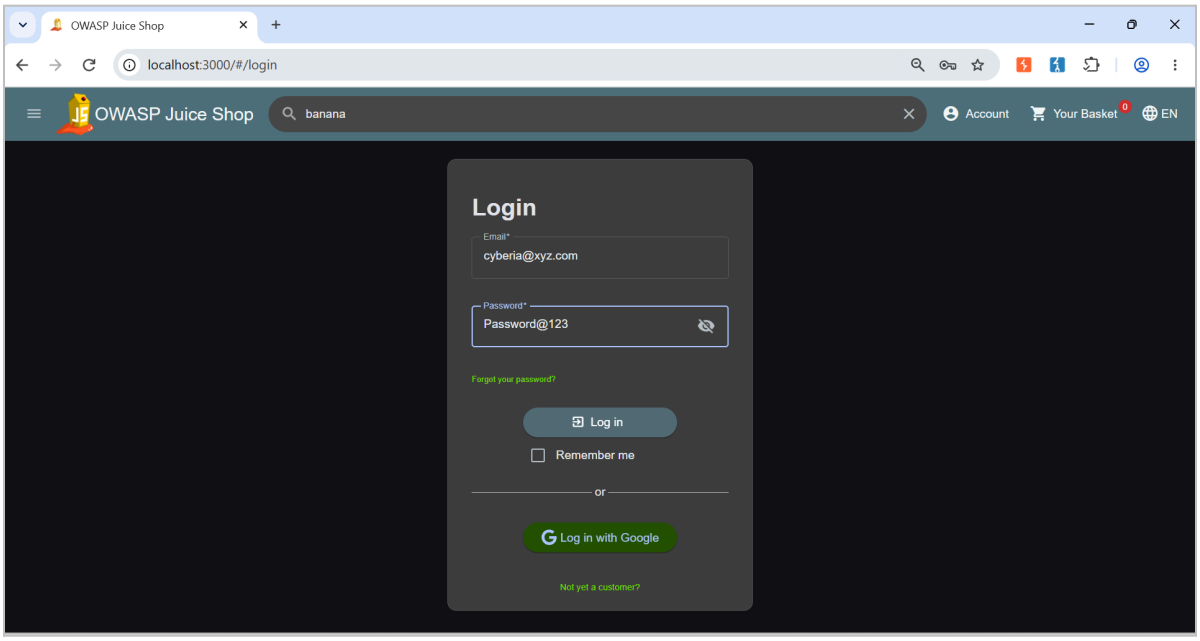


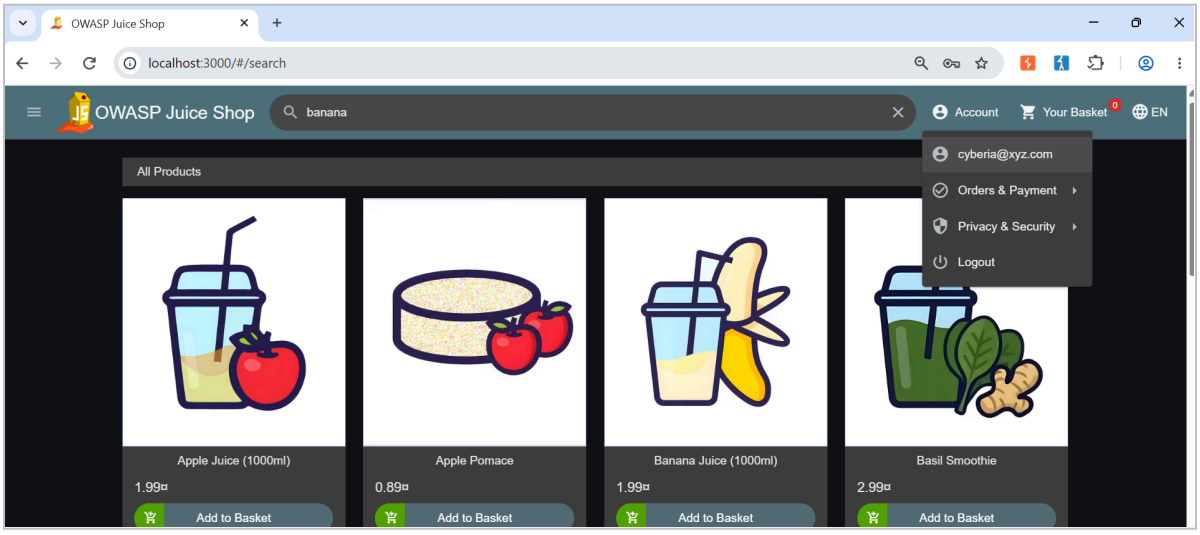
The registration request is also captured in Burp Suite.



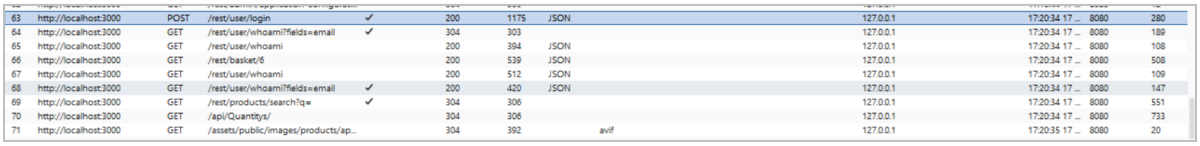


6.3 Login request and response

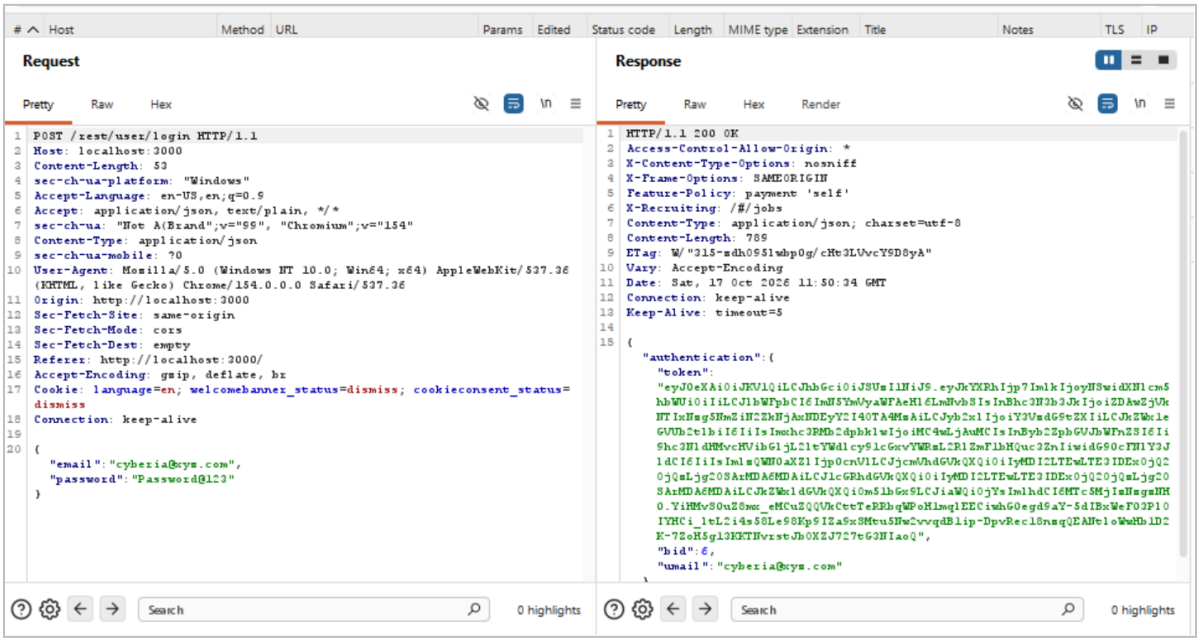




The login request is captured in Burp Suite.

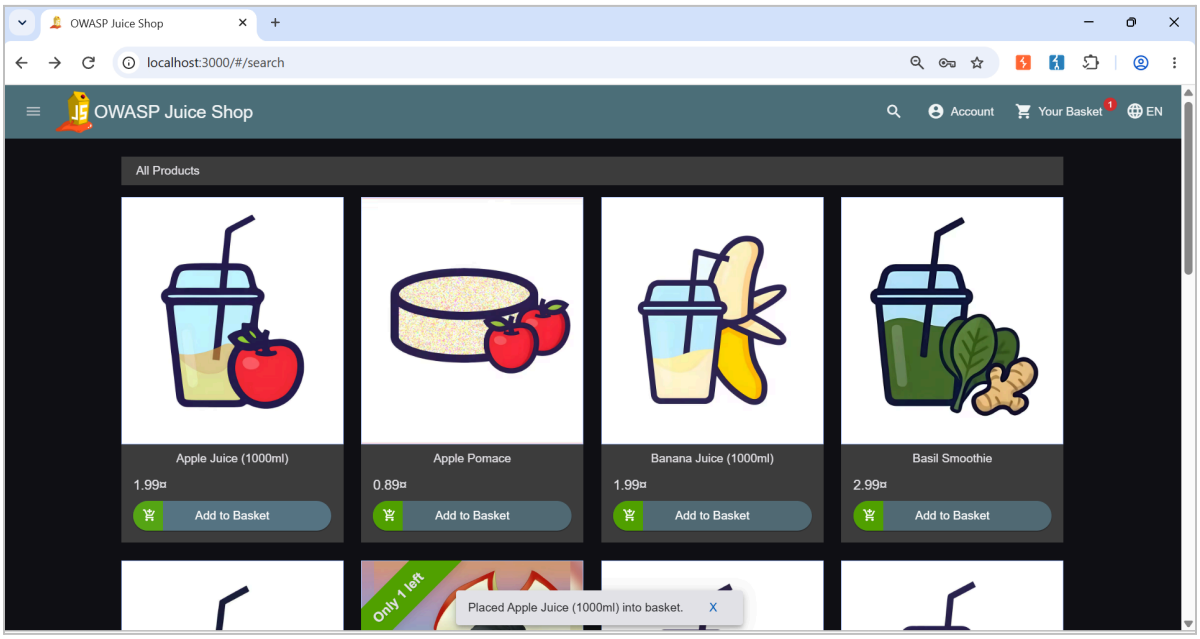


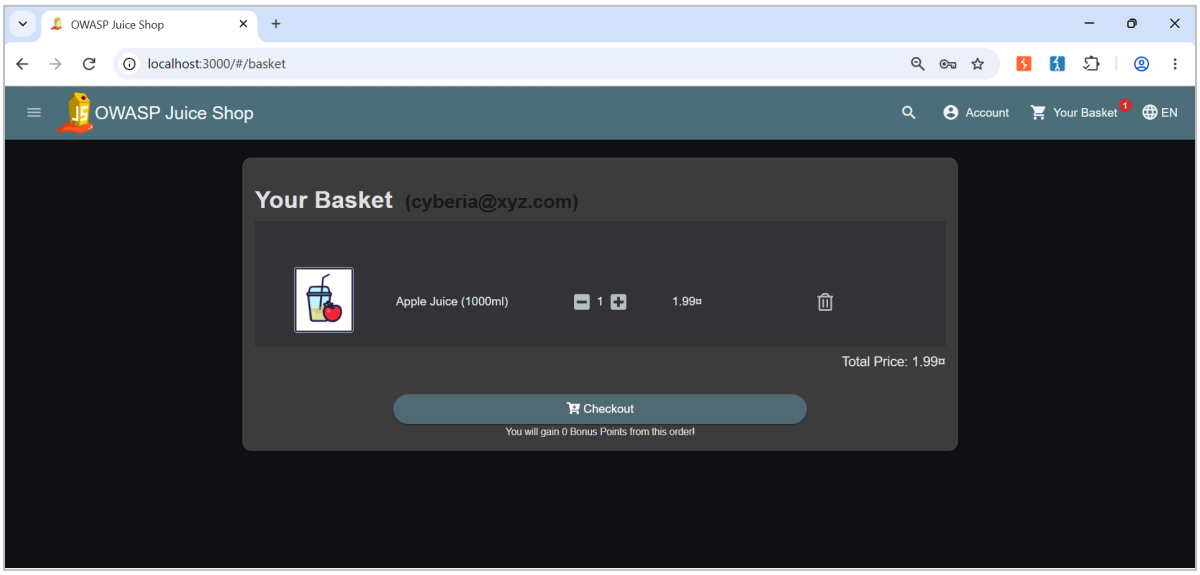
63	http://localhost:3000	POST	/rest/user/login	✓	200	1175	JSON		127.0.0.1	17:20:34 17 ...	8080	280
64	http://localhost:3000	GET	/rest/user/whoami?fields=email	✓	304	303			127.0.0.1	17:20:34 17 ...	8080	189
65	http://localhost:3000	GET	/rest/user/whoami		200	394	JSON		127.0.0.1	17:20:34 17 ...	8080	108
66	http://localhost:3000	GET	/rest/basket/6		200	539	JSON		127.0.0.1	17:20:34 17 ...	8080	508
67	http://localhost:3000	GET	/rest/user/whoami		200	512	JSON		127.0.0.1	17:20:34 17 ...	8080	109
68	http://localhost:3000	GET	/rest/user/whoami?fields=email	✓	200	420	JSON		127.0.0.1	17:20:34 17 ...	8080	147
69	http://localhost:3000	GET	/rest/products/search?q=	✓	304	306			127.0.0.1	17:20:34 17 ...	8080	551
70	http://localhost:3000	GET	/api/Quantitys/		304	306			127.0.0.1	17:20:34 17 ...	8080	733
71	http://localhost:3000	GET	/assets/public/images/products/ap...		304	392		avif	127.0.0.1	17:20:35 17 ...	8080	20

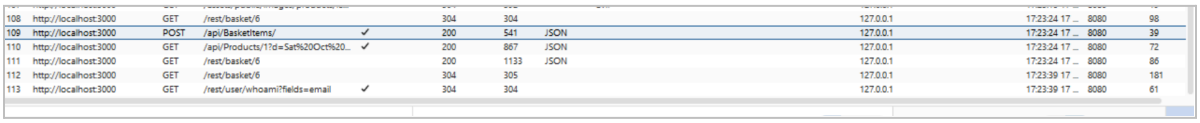


6.4 Cart-related requests and quantity

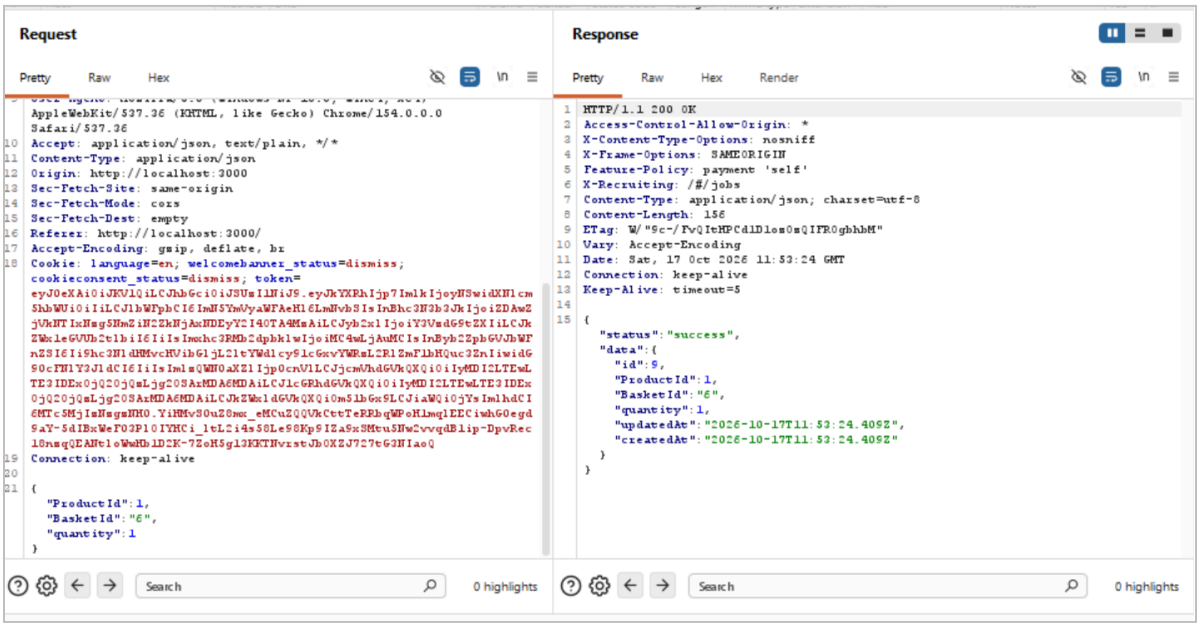
A product was added to the cart:



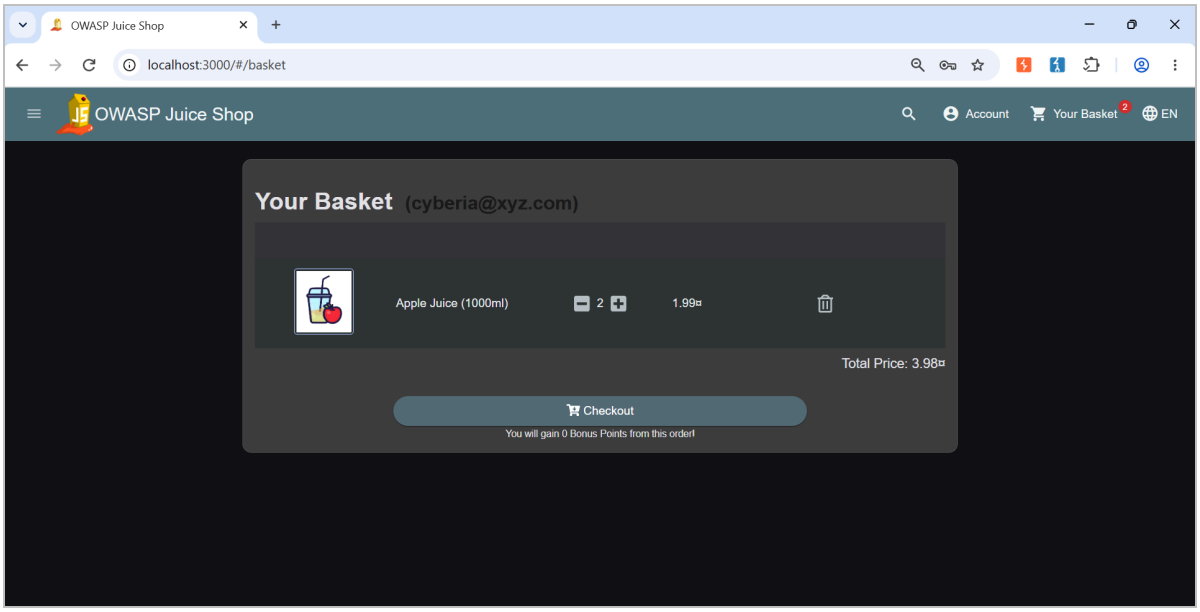


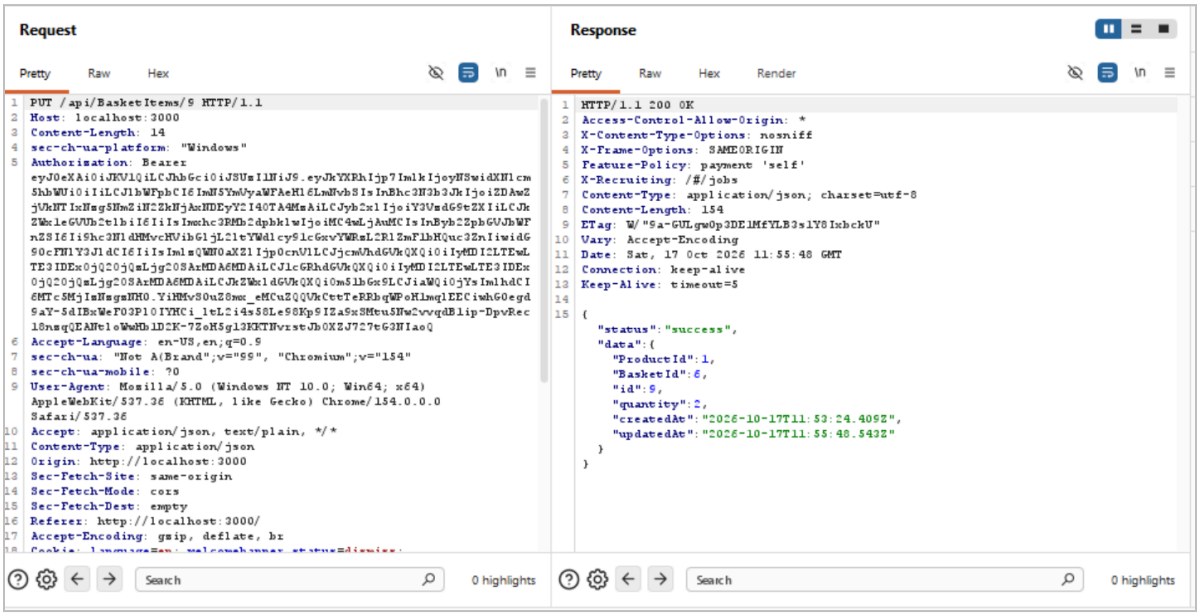


108	http://localhost:3000	GET	/rest/basket/6		304	304		127.0.0.1	17:23:24 17 ...	8080	98
109	http://localhost:3000	POST	/api/BasketItems/	✓	200	541	JSON	127.0.0.1	17:23:24 17 ...	8080	39
110	http://localhost:3000	GET	/api/Products/1?d=Sat%20Oct%20...	✓	200	867	JSON	127.0.0.1	17:23:24 17 ...	8080	72
111	http://localhost:3000	GET	/rest/basket/6		200	1133	JSON	127.0.0.1	17:23:24 17 ...	8080	86
112	http://localhost:3000	GET	/rest/basket/6		304	305		127.0.0.1	17:23:39 17 ...	8080	181
113	http://localhost:3000	GET	/rest/user/whoami?fields=email	✓	304	304		127.0.0.1	17:23:39 17 ...	8080	61



The quantity of the product was then increased in the cart:





These are the basic functions of OWASP Juice Shop whose requests and responses can be observed in Burp Suite. They form the basis for the security tests that follow.

7. Security Testing

Each test below lists the procedure followed, what was to be recorded, and the observations made. A different response, a missing header or an error message is treated as a lead only; a finding is reported as a vulnerability only when a real security impact is established.

Test 1 – Authentication and Session Management

Procedure

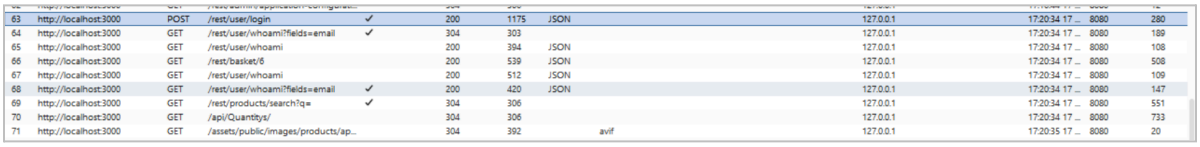
1. Log in with your test account.
2. Identify the login request in HTTP history.
3. Inspect the response for authentication tokens or session cookies.
4. Log out and try to access a protected feature again.

Record: Whether logout prevents further authenticated access, and whether tokens or sensitive details are exposed unexpectedly.

Observations

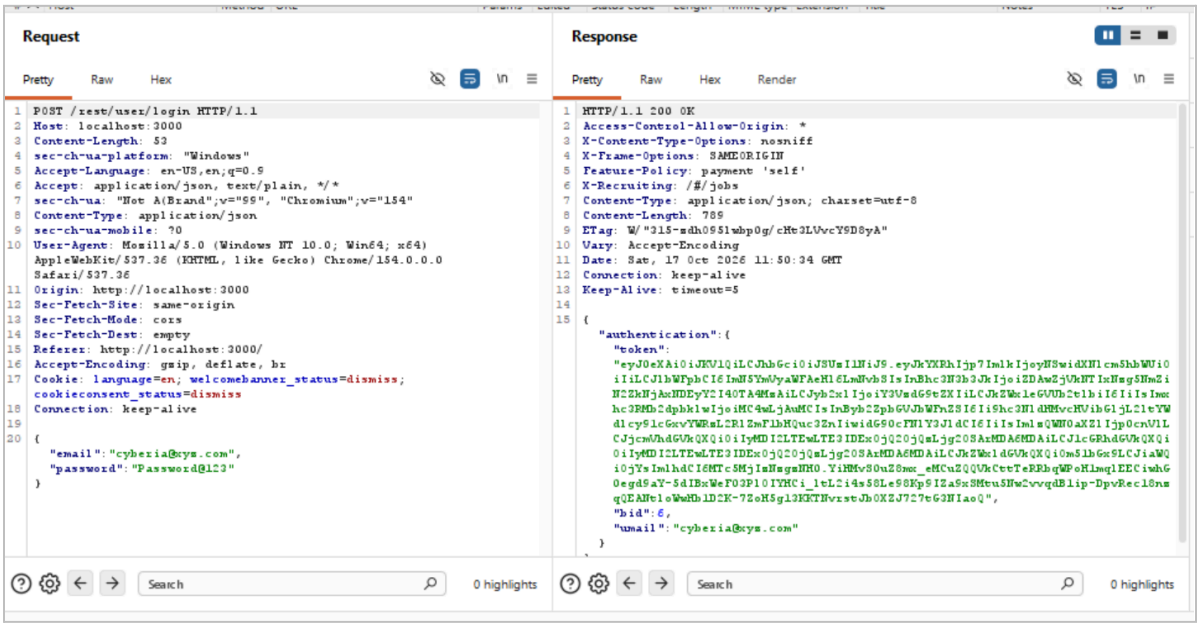
(a) Login

The test account was logged in with the correct username and password; the login request is shown below.



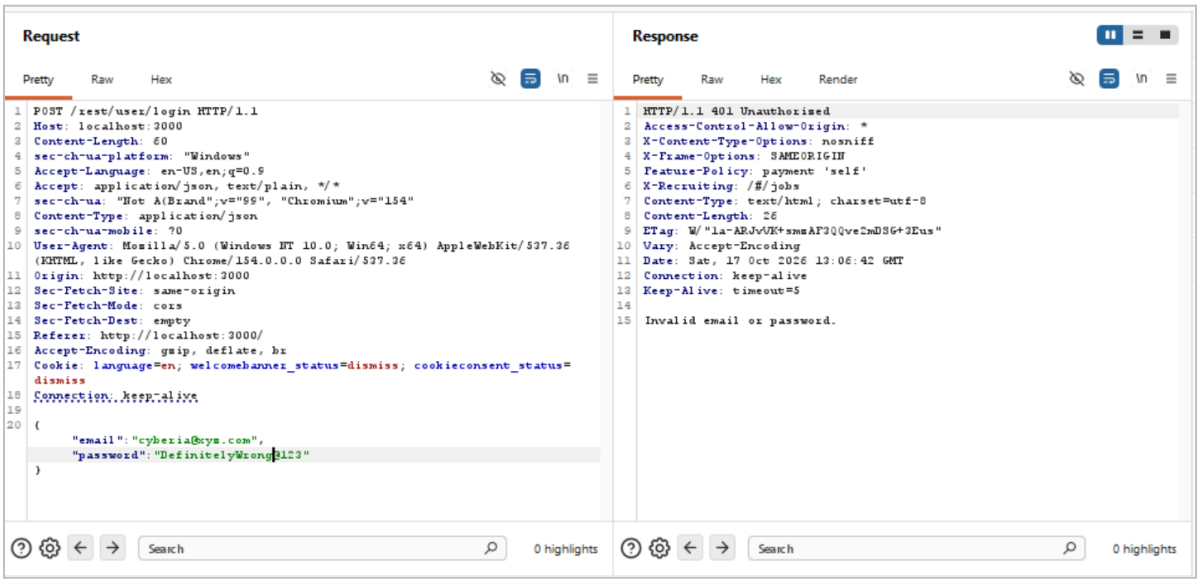
63	http://localhost:3000	POST	/rest/user/login	✓	200	1175	JSON		127.0.0.1	17:20:34 17 ...	8080	280
64	http://localhost:3000	GET	/rest/user/whoami?fields=email	✓	304	303			127.0.0.1	17:20:34 17 ...	8080	189
65	http://localhost:3000	GET	/rest/user/whoami		200	394	JSON		127.0.0.1	17:20:34 17 ...	8080	108
66	http://localhost:3000	GET	/rest/basket/6		200	539	JSON		127.0.0.1	17:20:34 17 ...	8080	508
67	http://localhost:3000	GET	/rest/user/whoami		200	512	JSON		127.0.0.1	17:20:34 17 ...	8080	109
68	http://localhost:3000	GET	/rest/user/whoami?fields=email	✓	200	420	JSON		127.0.0.1	17:20:34 17 ...	8080	147
69	http://localhost:3000	GET	/rest/products/search?q=	✓	304	306			127.0.0.1	17:20:34 17 ...	8080	551
70	http://localhost:3000	GET	/api/Quantitys/		304	306			127.0.0.1	17:20:34 17 ...	8080	733
71	http://localhost:3000	GET	/assets/public/images/products/ap...		304	392		avif	127.0.0.1	17:20:35 17 ...	8080	20

The response returned 200 OK together with an authentication token. This shows that the login was successful and that the server issued a token for the session. The response did not reveal any unexpected private information.



(b) Invalid credentials

The login request was sent to Repeater and the password was changed to an incorrect one. The following response was obtained:

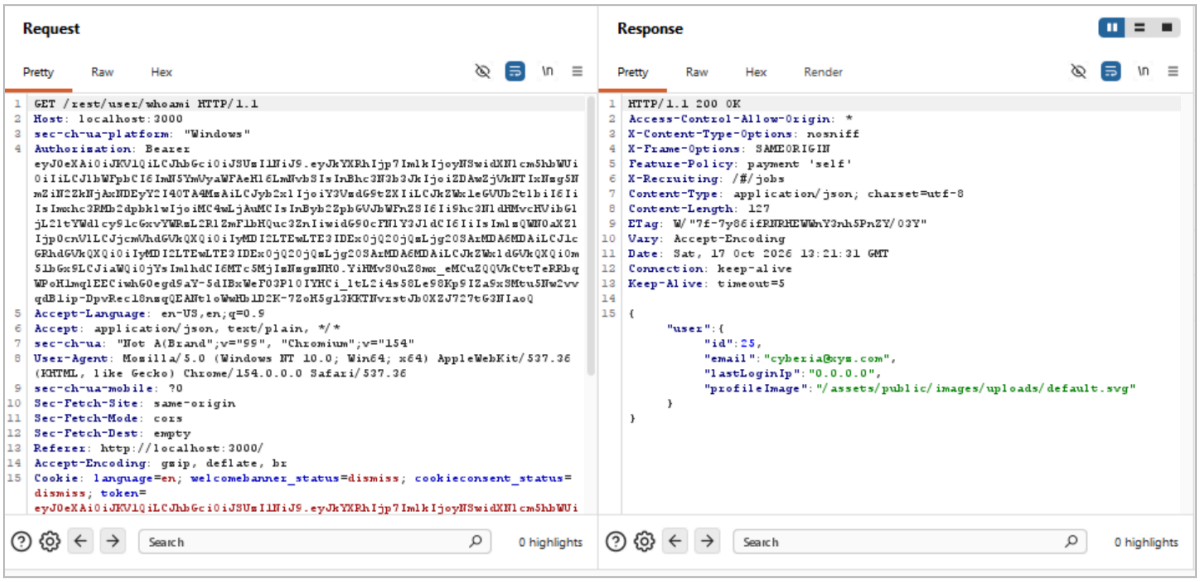


The login was rejected for the incorrect password, which is the expected security behaviour. No authenticated session was created and the server returned no internal error, so there is no sign of a serious vulnerability that needs further investigation. The login authentication is therefore not vulnerable in this test.

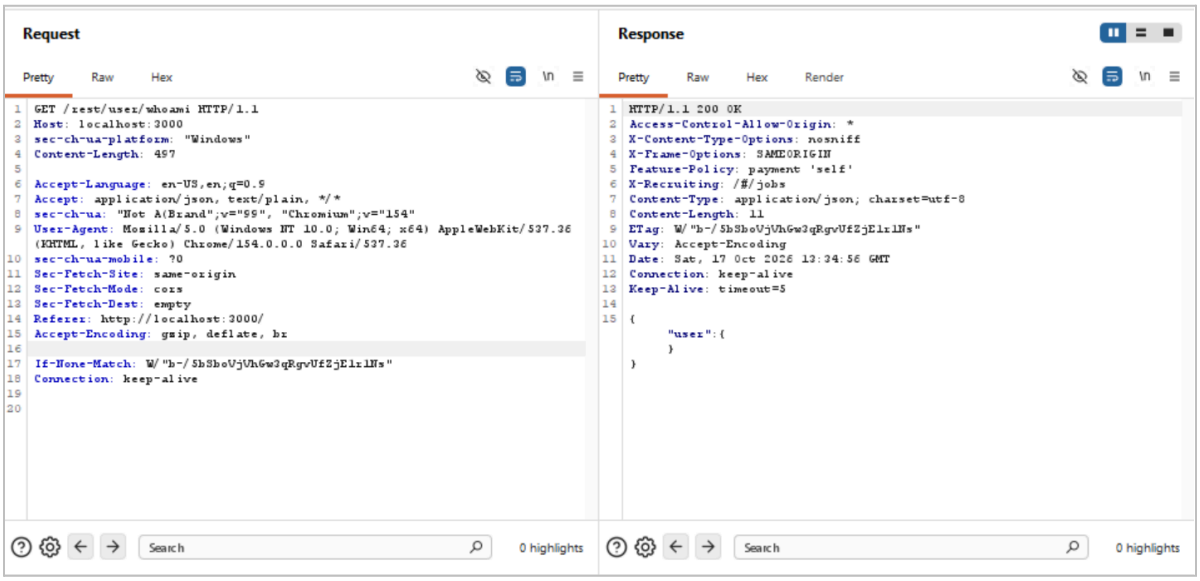
(c) Logout

After a successful login, the request returned 200 OK, and the response contained the user id and email.





The same request was repeated after logging out of Juice Shop:



- HTTP 200 OK: the server processed the request successfully at the HTTP level.
- "user": {}: the response no longer contains the user details seen before logout.
- No account information was returned, which suggests the endpoint does not reveal private profile data once authentication is removed.

Result: Logout stopped this endpoint from returning authenticated user details, and no tokens or sensitive details were exposed unexpectedly. Although the endpoint returns 200 OK, it returns an empty user object when unauthenticated, so this test does not demonstrate a broken access control vulnerability.

Test 2 – Broken Access Control

Procedure

1. Create two test accounts that you control.
2. Capture a request for an account-specific resource while logged in as Account A.
3. Repeat the same normal action as Account B.
4. Compare what each account is authorized to access.

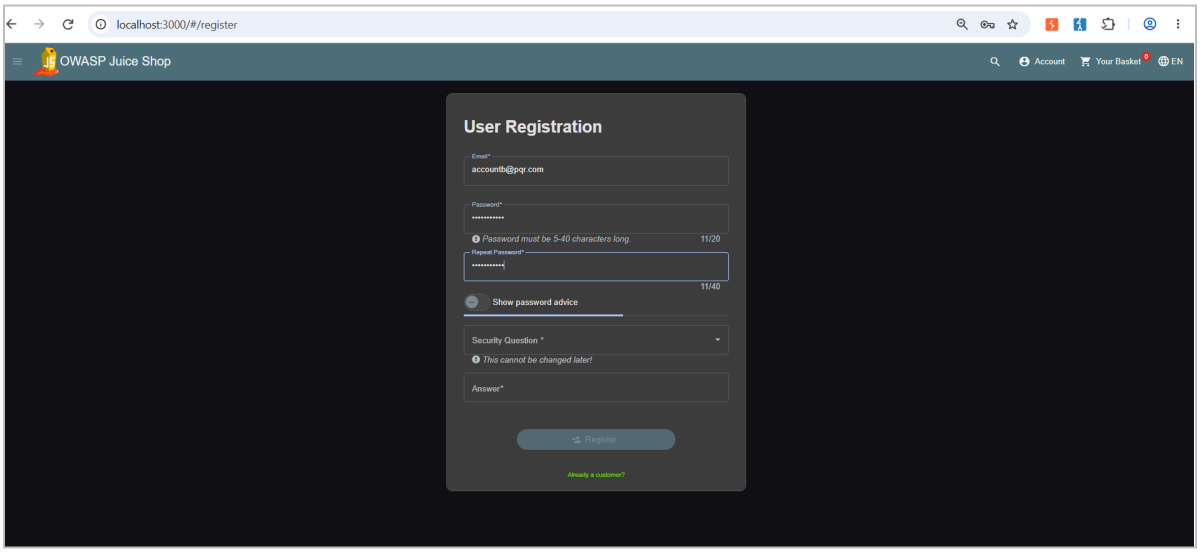
Record: Whether either account can access the other account's private information without permission. A different response alone is not proof of a vulnerability.

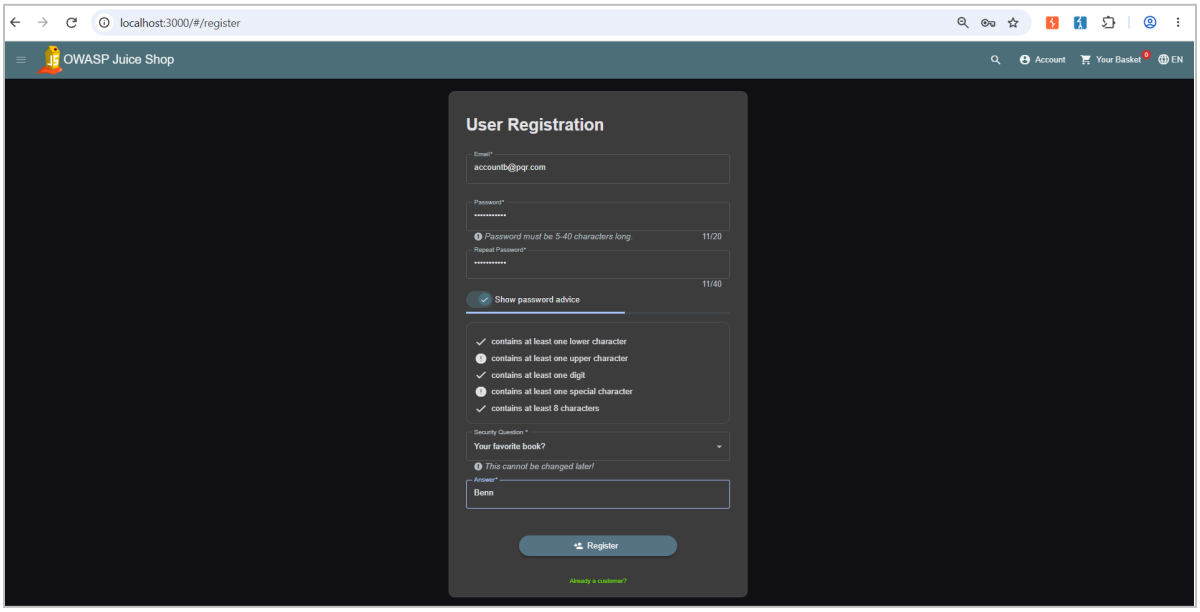
Two new test accounts, Account B and Account C, were created for this test and their profile pages were compared.

Observations

(a) Creating Account B

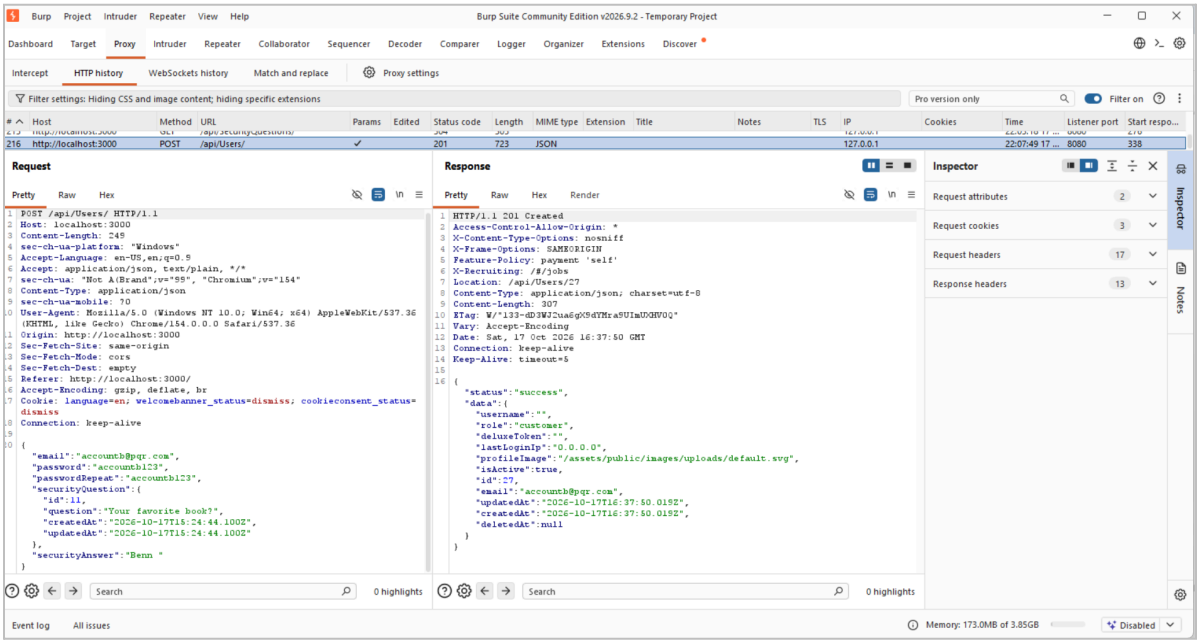
Account B was registered with the email accountb@pqr.com.

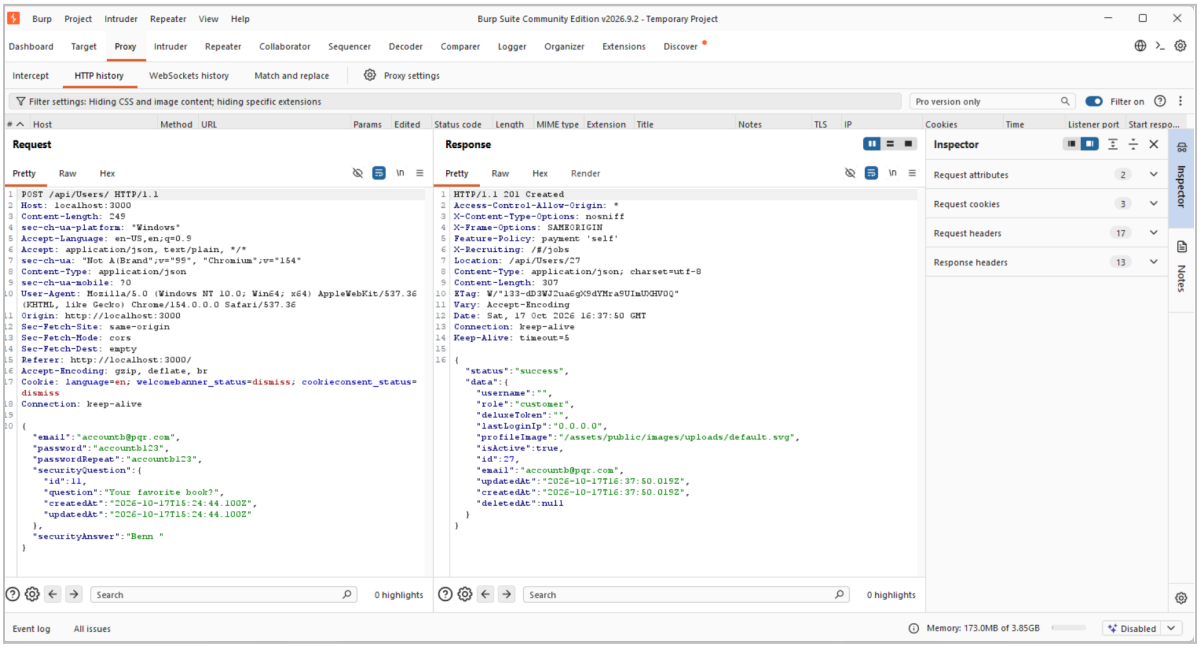




Password used: accountb123 (test data only).

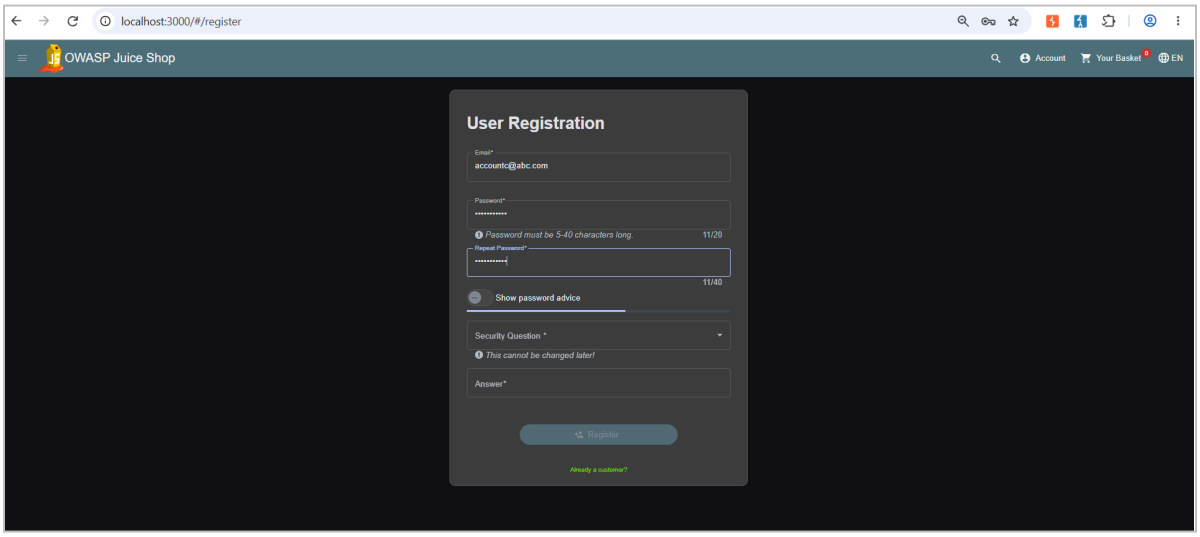
In Burp Suite, under Proxy > HTTP history, the POST request for the registration was found and its request and response were captured.

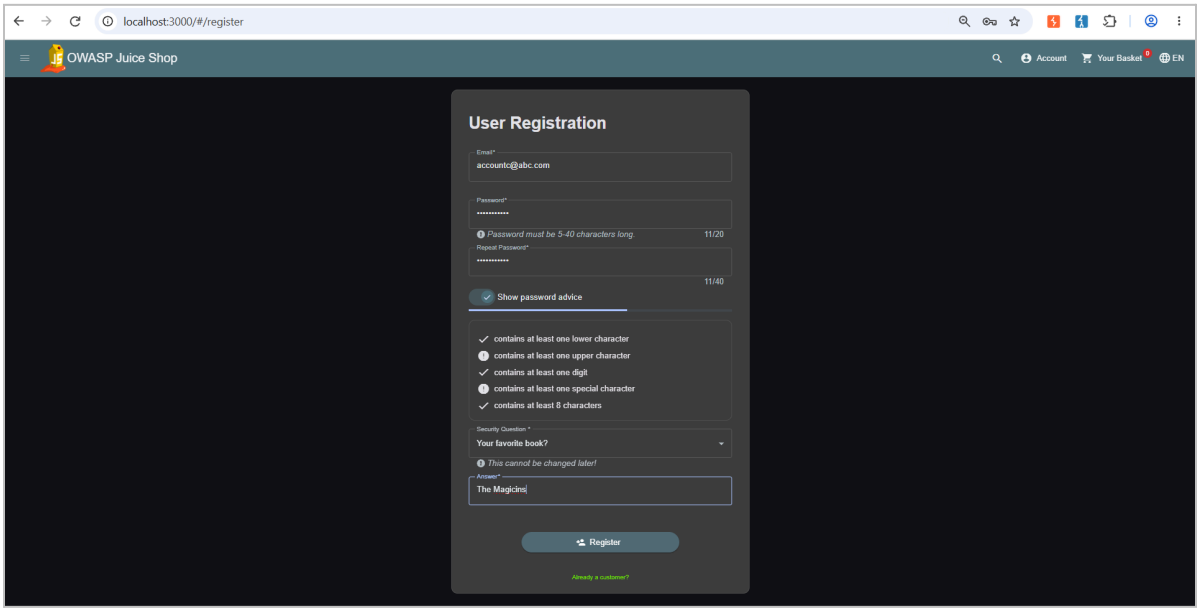




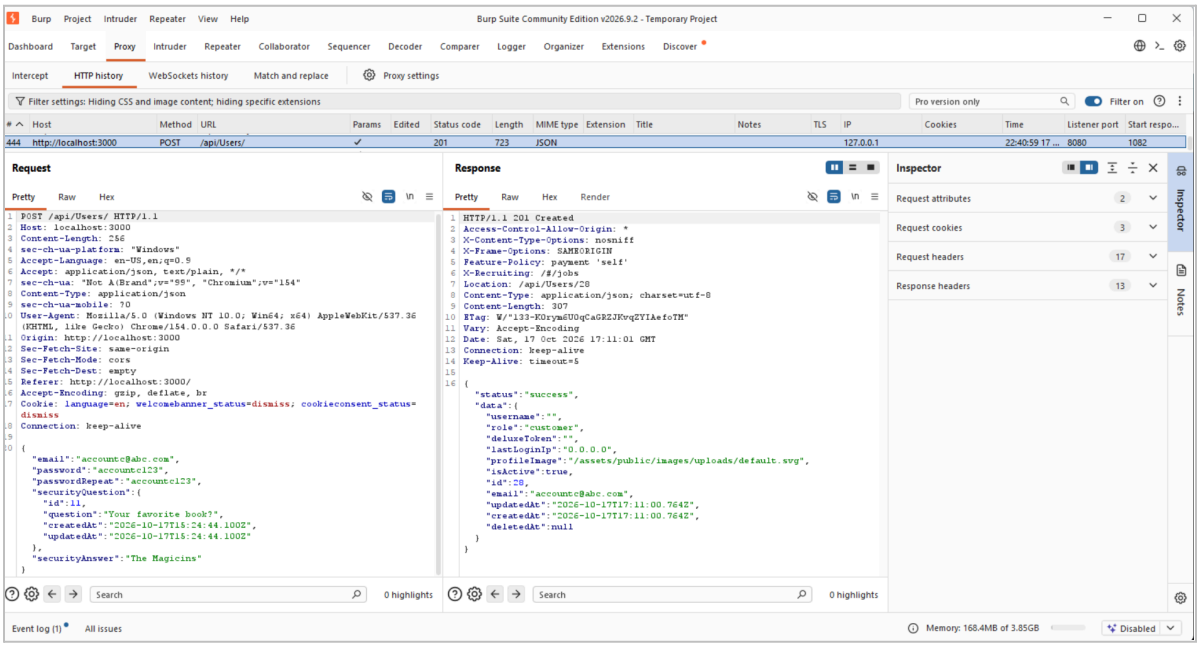
(b) Creating Account C

Account C was registered with the email accountc@abc.com.



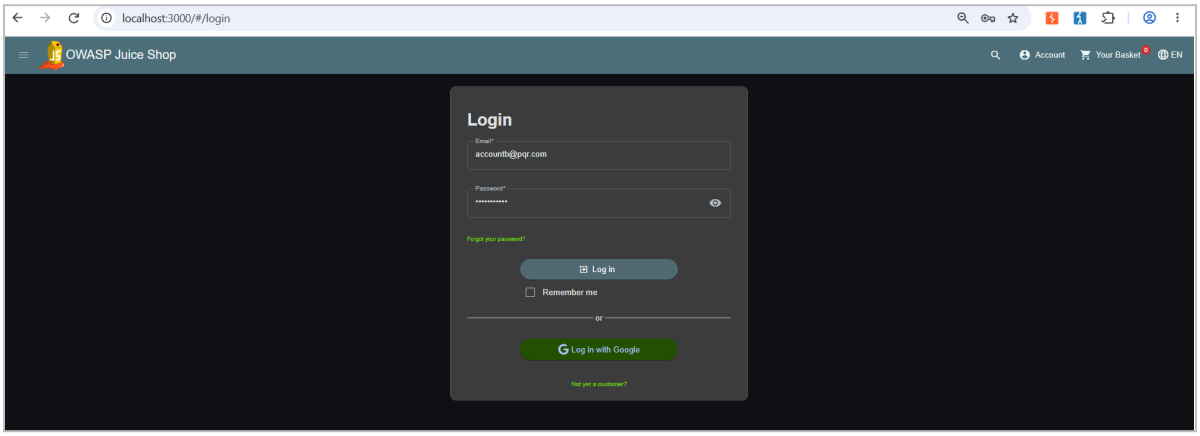


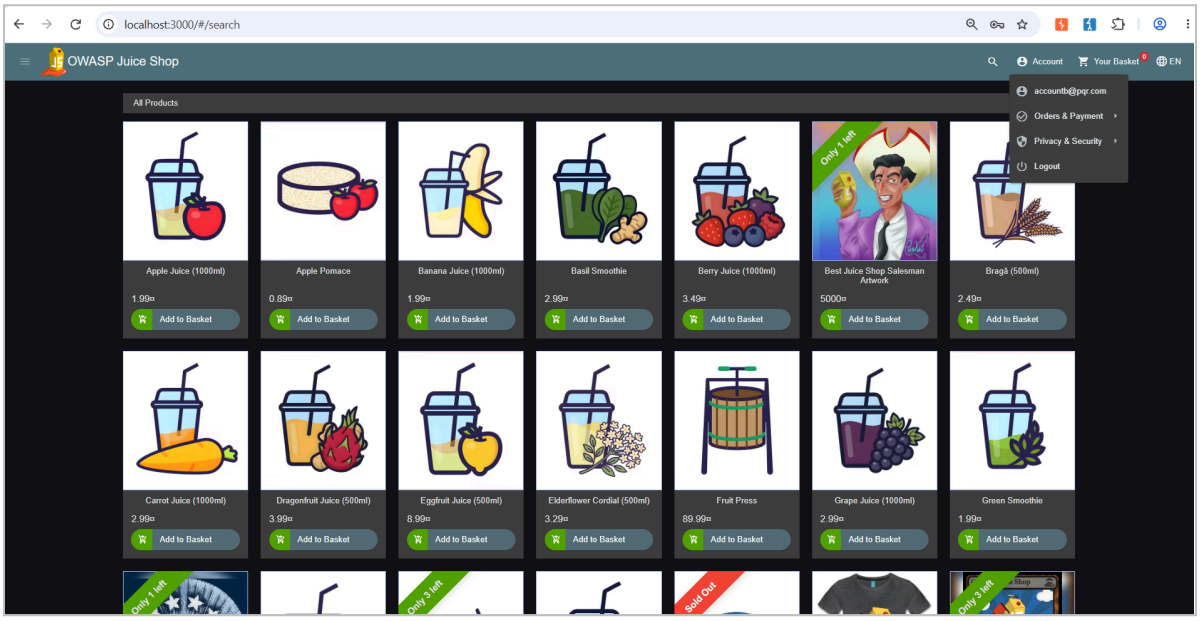
Password used: accountc123 (test data only).

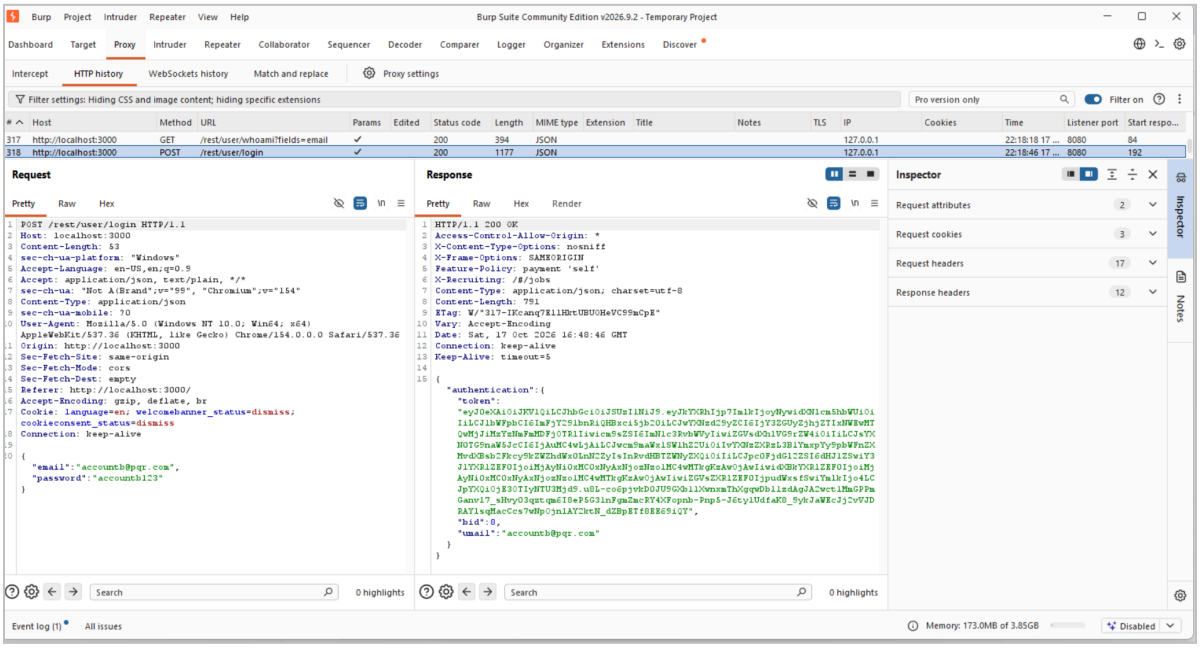


(c) Logging in and viewing the profile as Account B

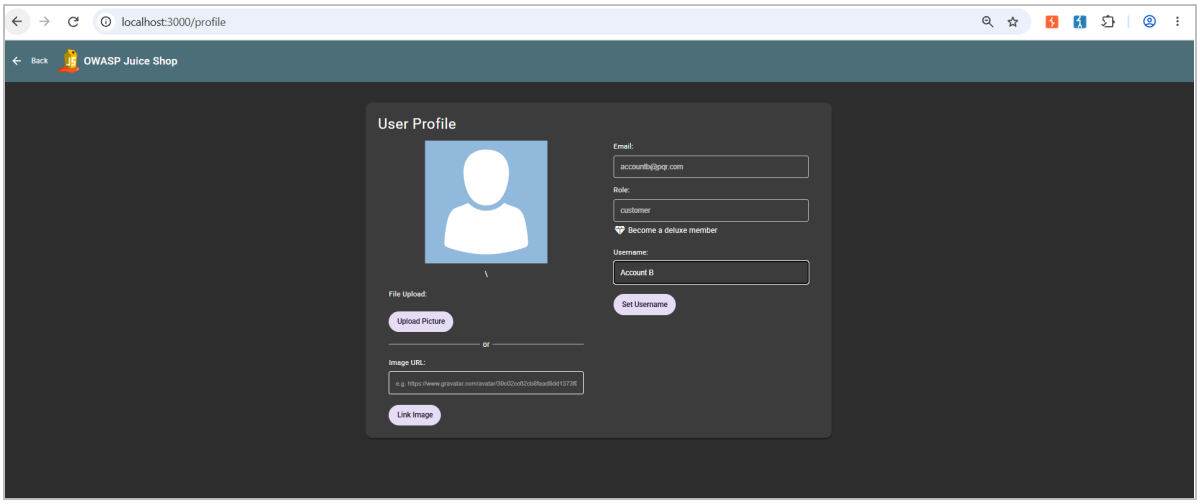
Step 1: Logged in as Account B.

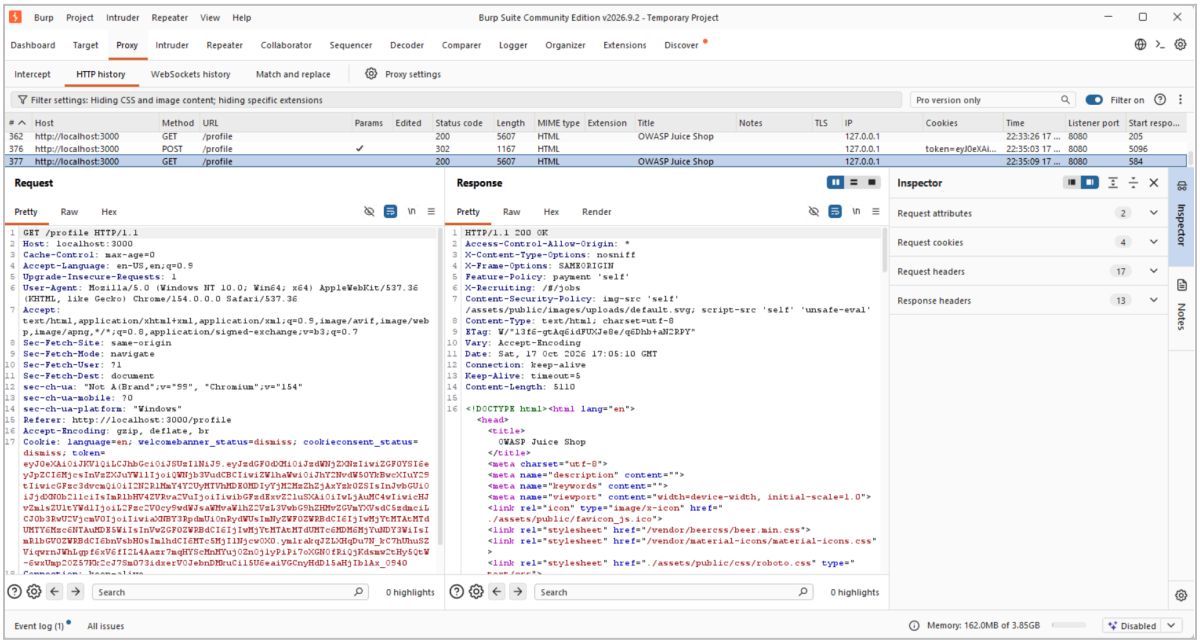


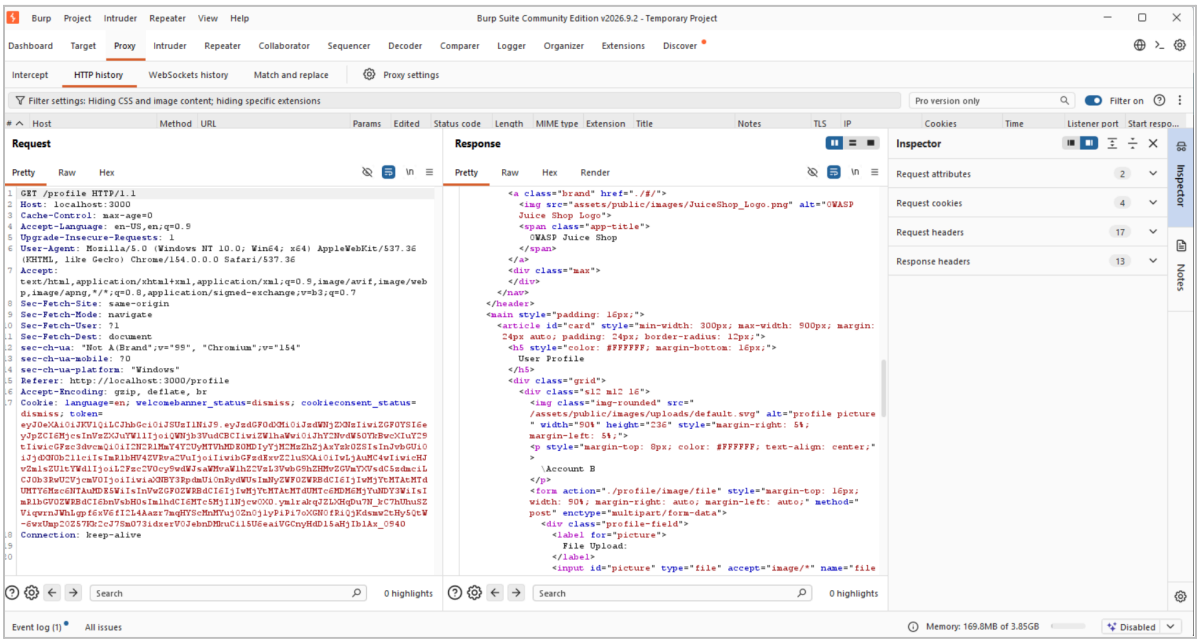


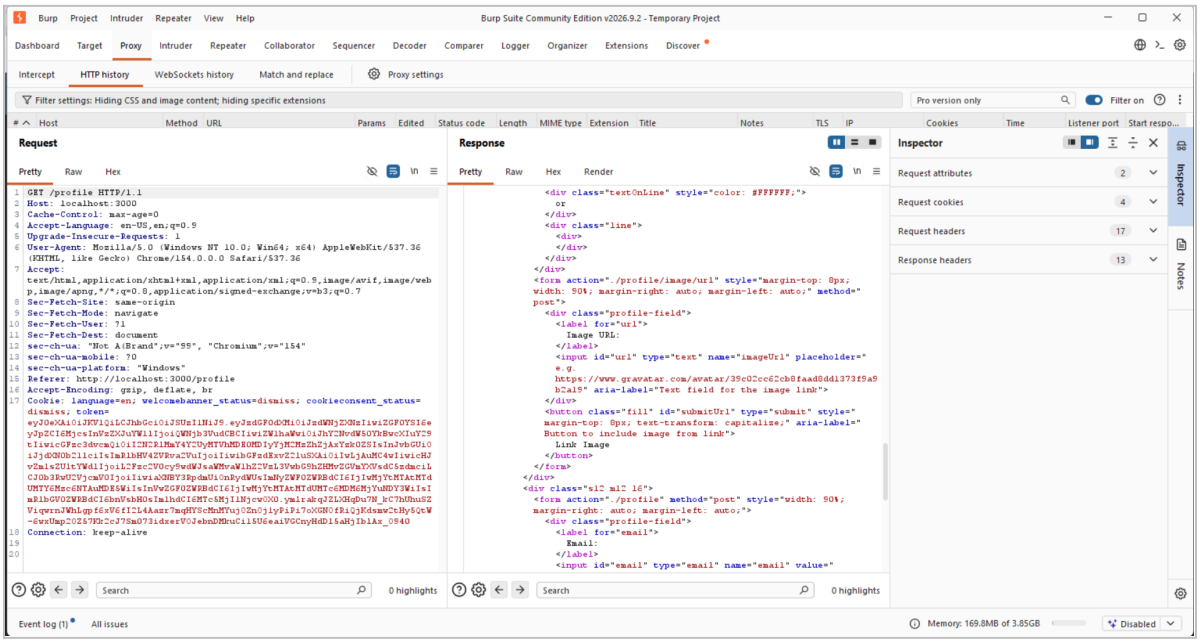


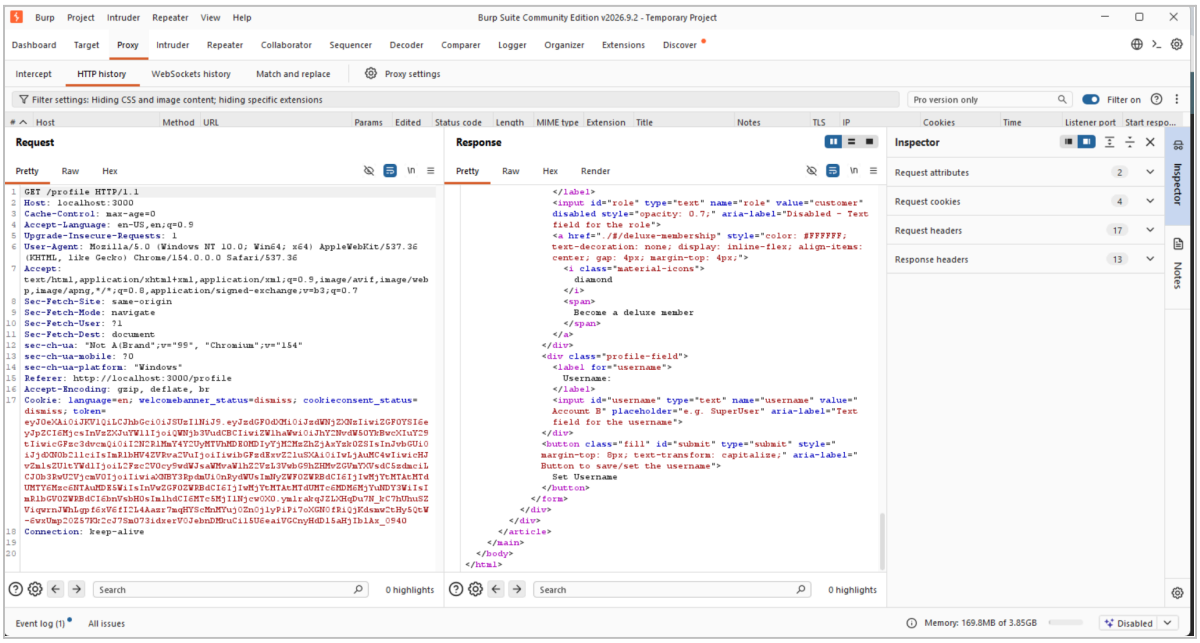
Account B profile page and the corresponding profile request and response in Burp Suite:




Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.9.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Filter settings: Hiding CSS and image content; hiding specific extensions
Request
Response
Inspector
Request attributes
Request cookies
Request headers
Response headers
Notes
Event log (1)
All issues
Memory: 162.0MB of 3.85GB
Disabled

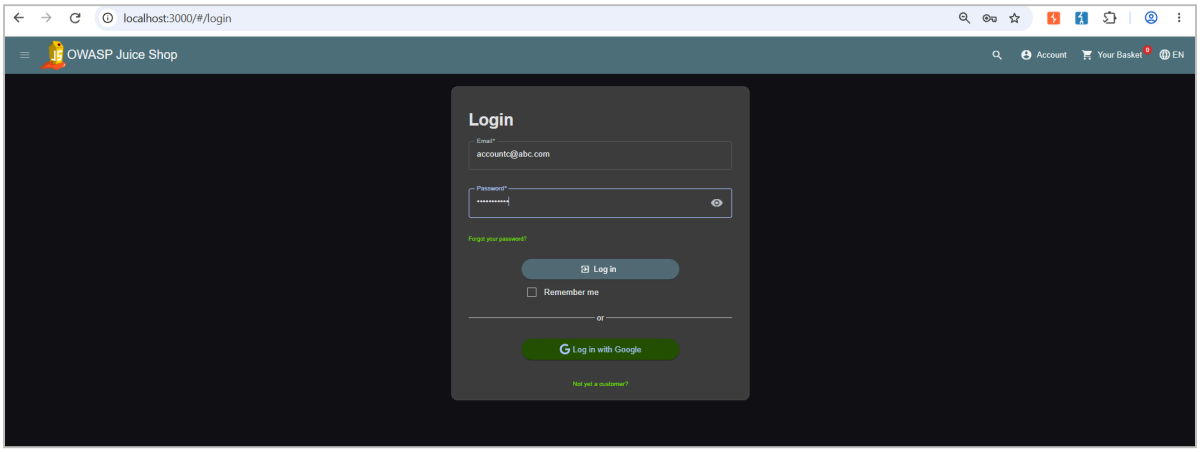

Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.9.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Filter settings: Hiding CSS and image content; hiding specific extensions
Request
Response
Inspector
Request attributes
Request cookies
Request headers
Response headers
Notes
Event log (1)
All issues
Memory: 169.8MB of 3.85GB
Disabled

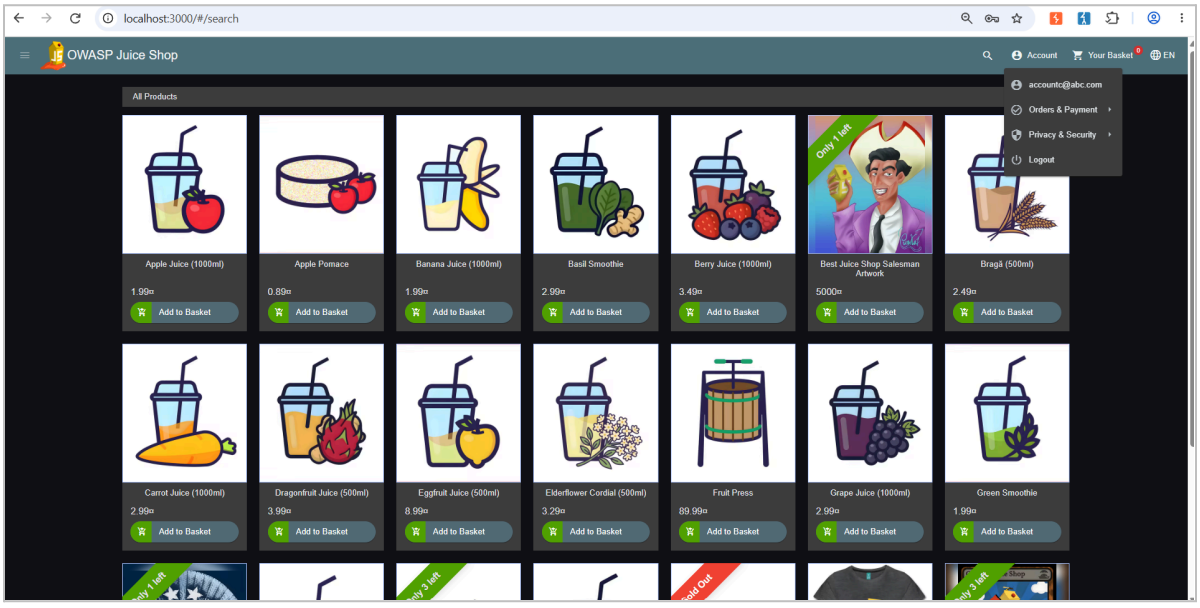

Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.9.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Request
Response
Inspector
Request attributes
Request cookies
Request headers
Response headers
Event log (1) All issues
Memory: 169.8MB of 3.85GB
Disabled

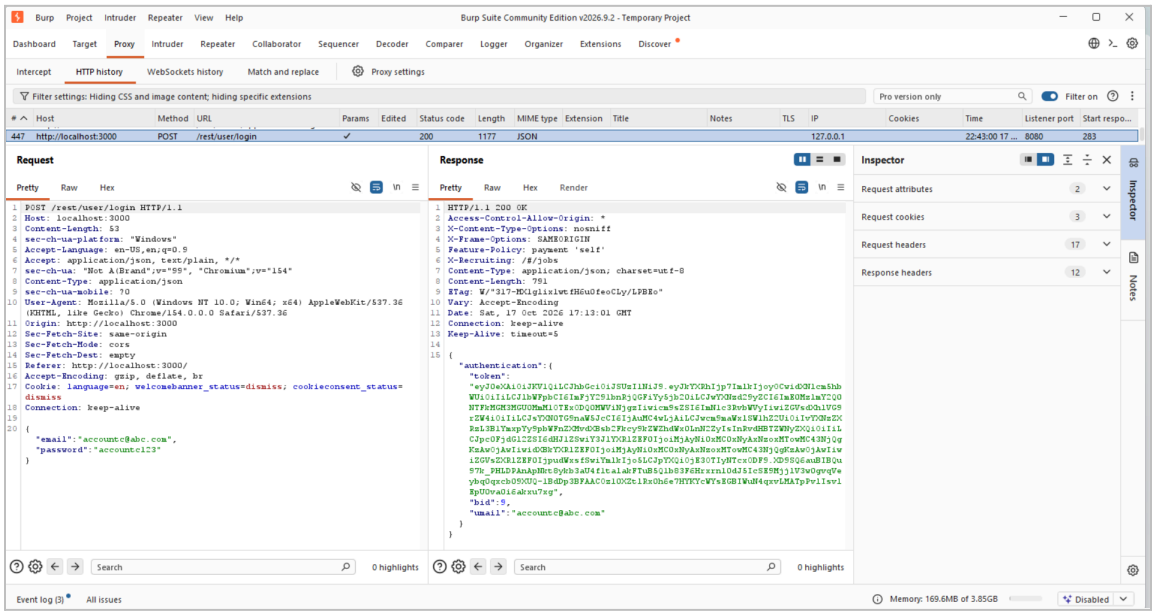

Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.9.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Request
Response
Inspector
Request attributes
Request cookies
Request headers
Response headers
Event log (1) All issues
Memory: 169.8MB of 3.85GB
Disabled

(d) Logging in and viewing the profile as Account C

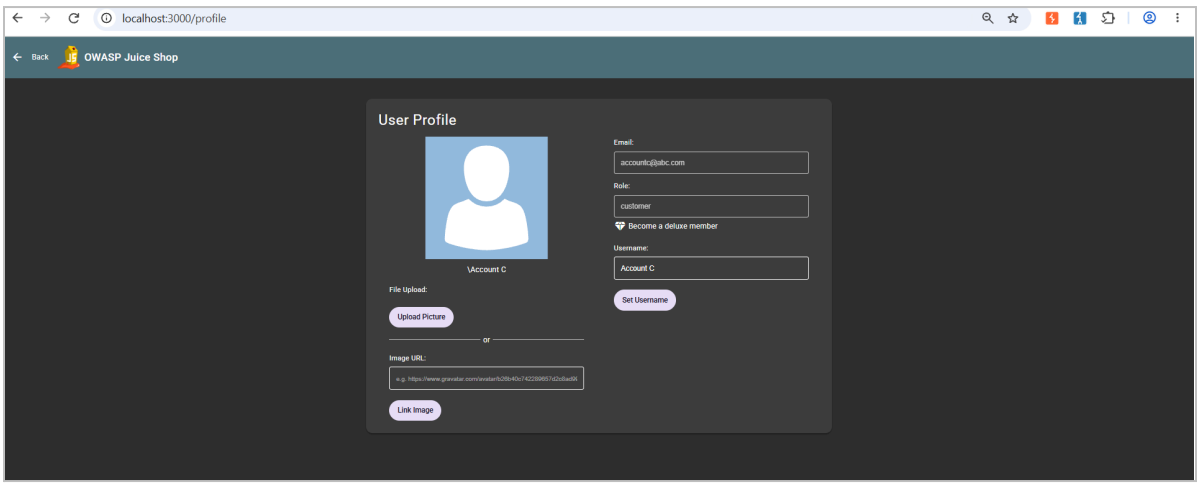
Step 2: Logged in as Account C.

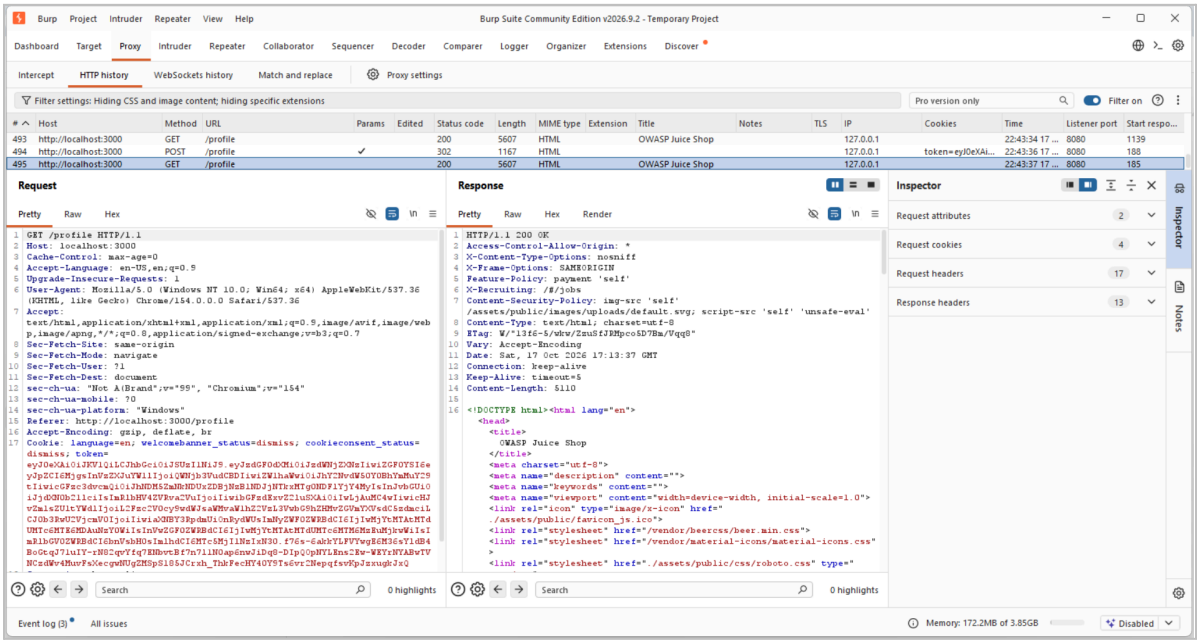


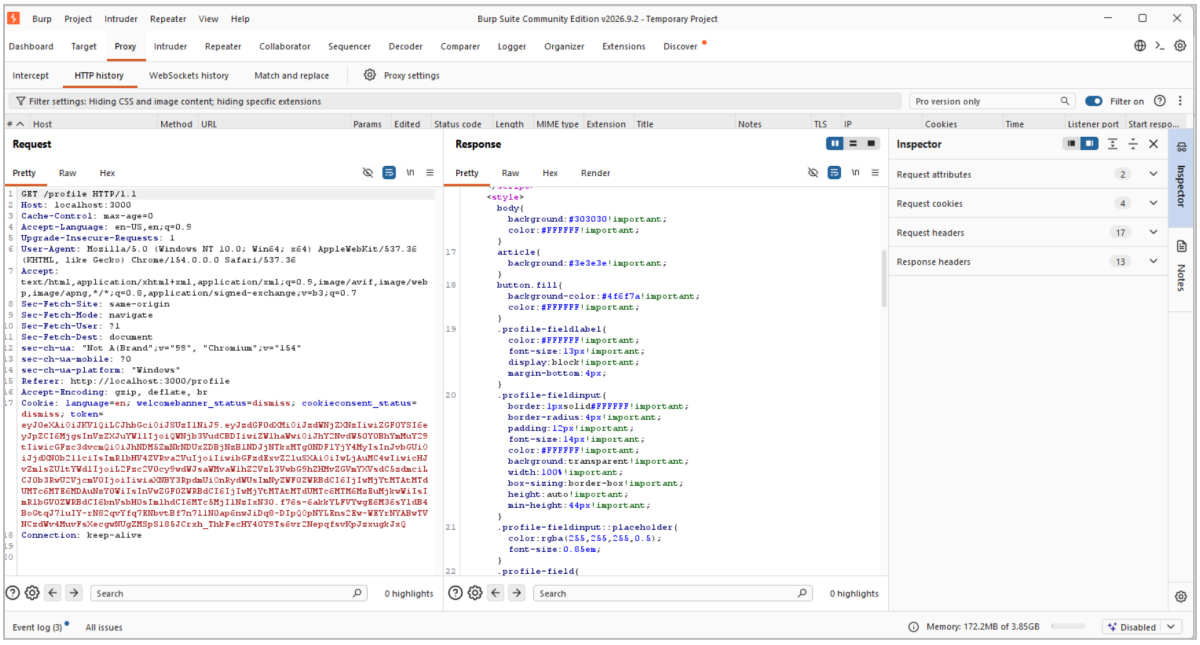


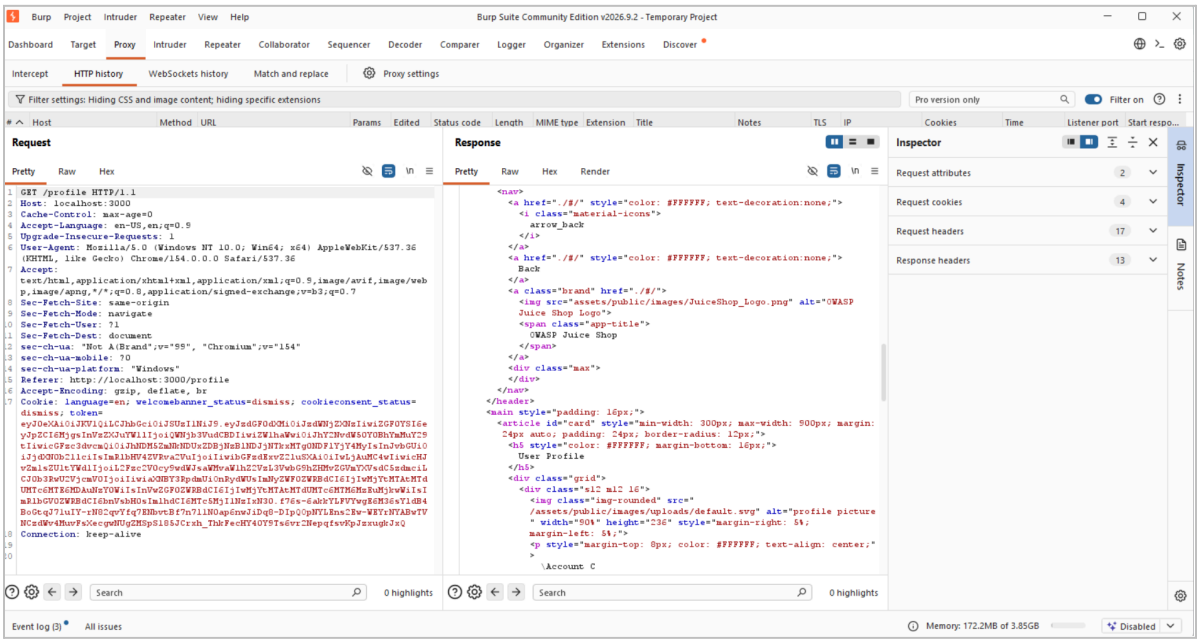


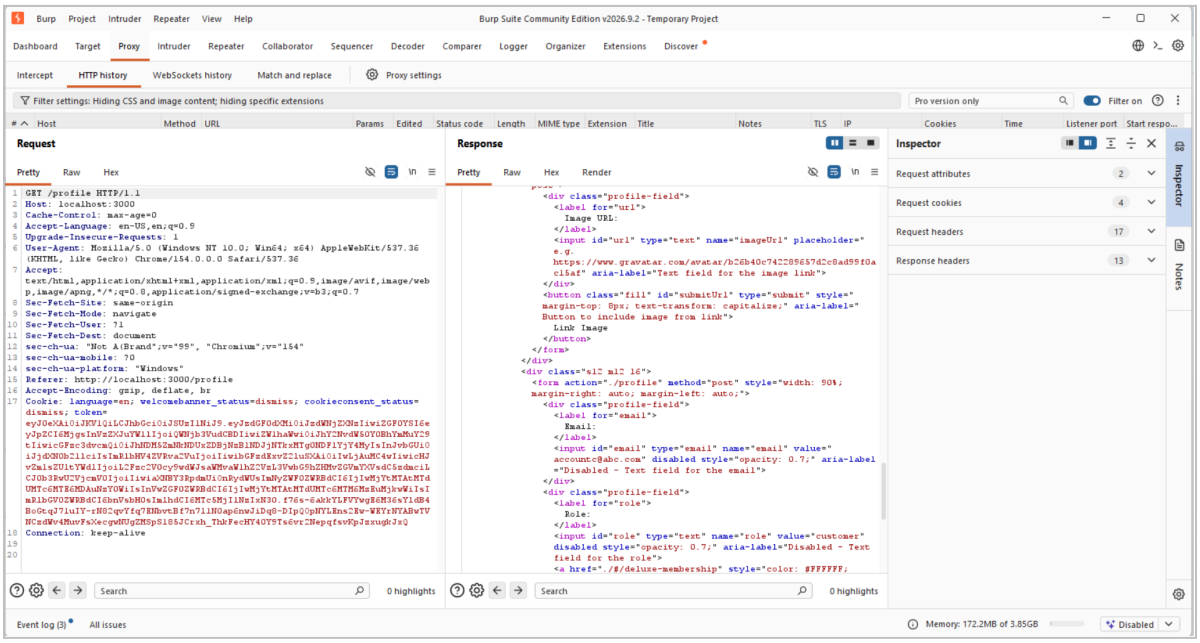
Account C profile page and the corresponding profile request and response in Burp Suite:

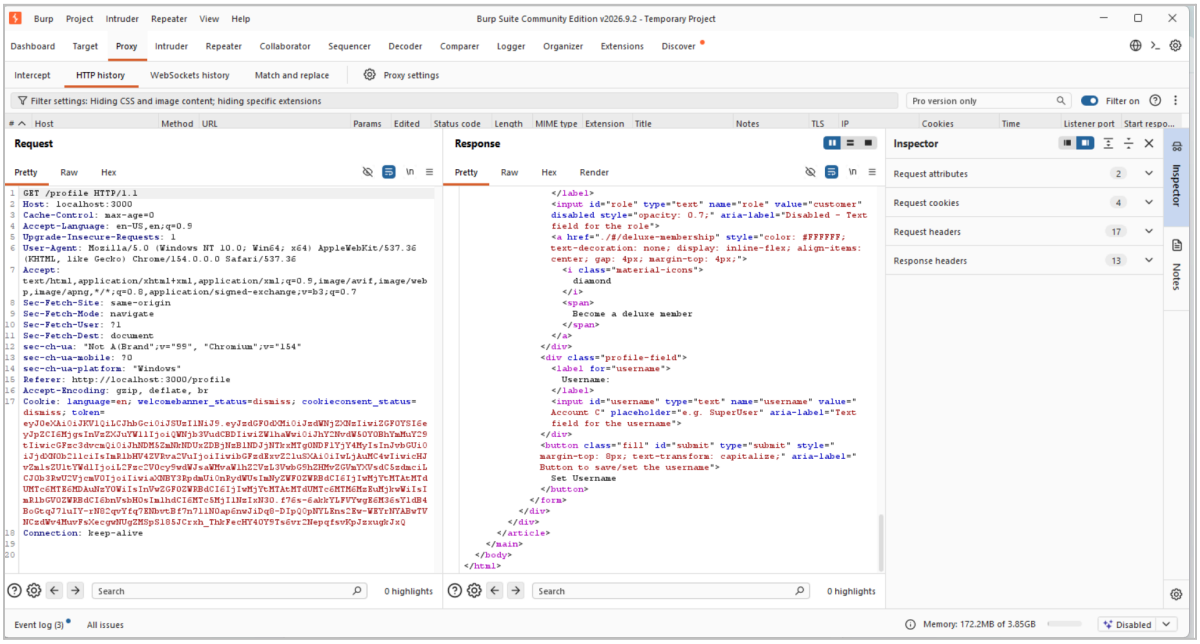



Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.9.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Filter settings: Hiding CSS and image content; hiding specific extensions
Request
Response
Inspector
Request attributes
Request cookies
Request headers
Response headers
Event log (3)
All issues
Memory: 172.2MB of 3.85GB
Disabled


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.9.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Filter settings: Hiding CSS and image content; hiding specific extensions
Request
Response
Inspector
Request attributes
Request cookies
Request headers
Response headers
Event log (3)
All issues
Memory: 172.2MB of 3.85GB
Disabled


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.9.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Request
Response
Inspector
Pretty Raw Hex
Pretty Raw Hex Render
GET /profile HTTP/1.1
Host: localhost:3000
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
sec-ch-ua: "Not A(Brand";v="99", "Chromium";v="154"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Referer: http://localhost:3000/profile
Accept-Encoding: gzip, deflate, br
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; token=
Connection: keep-alive
Request attributes
Request cookies
Request headers
Response headers
Search
0 highlights
Event log (3)
All issues
Memory: 172.2MB of 3.85GB
Disabled


Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.9.2 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Intercept HTTP history WebSockets history Match and replace Proxy settings
Request
Response
Inspector
Pretty Raw Hex
Pretty Raw Hex Render
GET /profile HTTP/1.1
Host: localhost:3000
Cache-Control: max-age=0
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
sec-ch-ua: "Not A(Brand";v="99", "Chromium";v="154"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Referer: http://localhost:3000/profile
Accept-Encoding: gzip, deflate, br
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; token=
Connection: keep-alive
Request attributes
Request cookies
Request headers
Response headers
Search
0 highlights
Event log (3)
All issues
Memory: 172.2MB of 3.85GB
Disabled



(e) Comparison of the profile responses

Detail	Account B	Account C
Email	accountb@pqr.com	accountc@abc.com
Role	customer	customer
Username	Account B	Account C
HTTP status	200 OK	200 OK

- Each response displays the corresponding account's own email and username. Based on these responses, unauthorized access to another user's profile was not demonstrated.
- Account B displayed accountb@pqr.com with username Account B, while Account C displayed accountc@abc.com with username Account C. Both responses returned HTTP status 200 OK.
- No broken access control vulnerability was demonstrated by this profile comparison.
- The application should enforce server-side authorization checks on all protected resources and prevent unauthorized access to other users' information.

Result: Only the profile page was compared between the two accounts, so this result covers that resource only. No broken access control was demonstrated.

Test 3 – Input Validation

Procedure

1. Send a product-search or feedback request to Repeater.
2. Try an empty field, an unusually long value, or unexpected punctuation.
3. Click Send and inspect the response.
4. Check for stack traces, database errors, unexpected data, or unsafe reflection of input.

Record: The input, response, expected behavior, and actual behavior. Suspicious results are treated as leads until a real security impact is established.

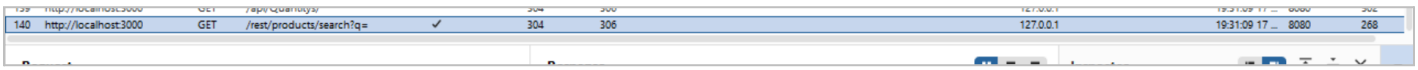
Summary of inputs tested

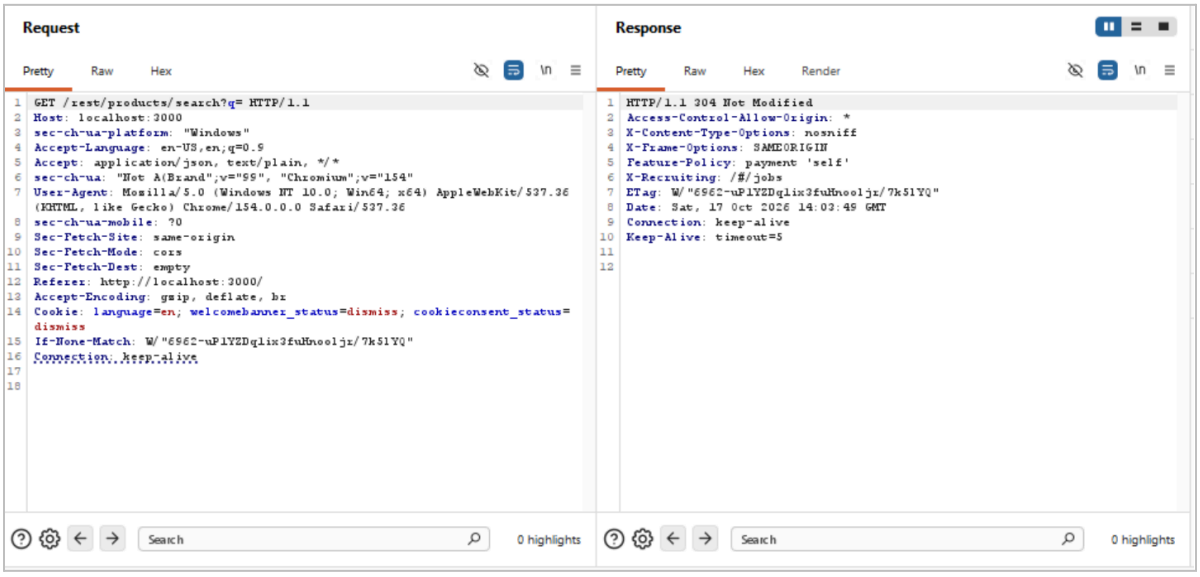
Input	Response	Expected behaviour	Actual behaviour
Empty search value	200 OK, normal empty response	Empty or normal response	As expected; no vulnerability shown
Long search value	200 OK, "status": "success", "data": []	Input handled safely	As expected; no error disclosure
Unexpected punctuation	HTTP 500 with SQLITE_ERROR and SQL query in the body	Input handled safely; generic error only	Database error and SQL query disclosed

Observations

(a) Empty search

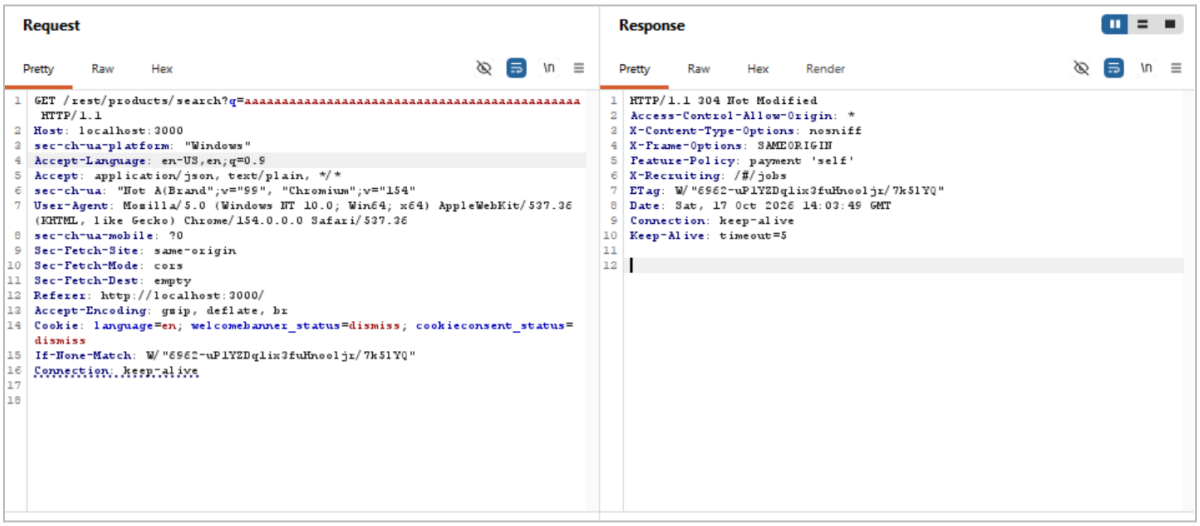
A search request was captured in Burp Suite.



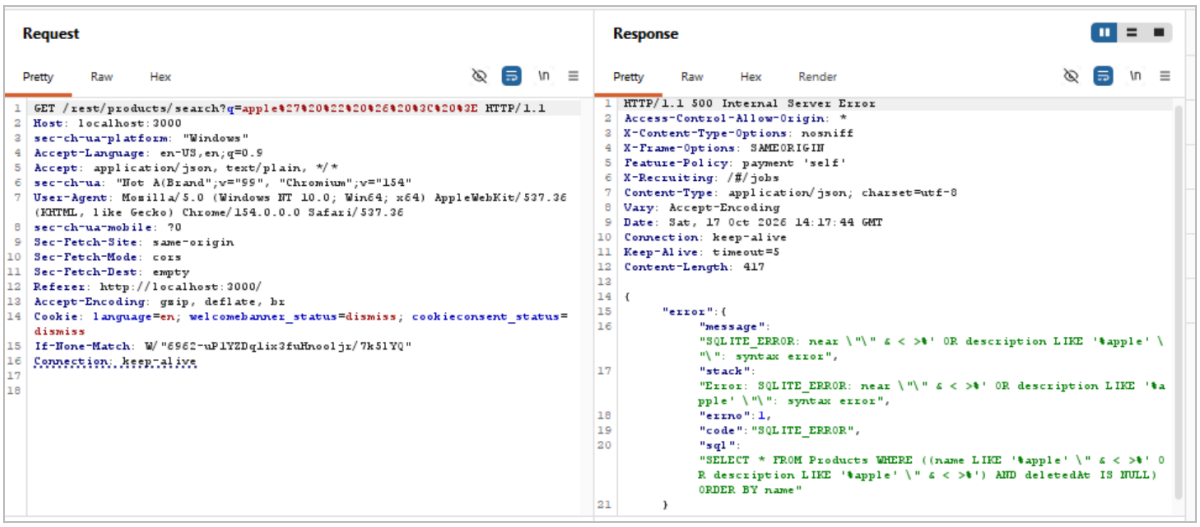


Juice Shop itself sends an empty search query here. An empty search returning no results, or a normal empty response, is generally acceptable and does not prove a vulnerability.

(b) Long search value


Request
Pretty
Raw
Hex
GET /rest/products/search?q=aaaa

(c) Unexpected punctuation



- HTTP 500 Internal Server Error: the server failed while processing the search input. A robust application should handle unexpected input without exposing internal database errors.
- The response contains SQLITE_ERROR and a SQL syntax error. This reveals that the endpoint uses an SQLite database and that the search input affects how the query is parsed.
- The sql field reveals the query structure, including the product name and description search conditions. This is sensitive implementation detail that should not normally be sent to a client.

In summary, the server returned HTTP 500 and disclosed SQLITE_ERROR, a stack trace, the SQL query and SQLite error details.

Expected result: Unexpected search input should be handled safely. The application should not expose SQL statements, database errors or stack traces to the client.

Potential impact: Detailed error disclosure can help an attacker understand the application's database and query structure. If unsafe SQL construction is confirmed to permit query manipulation, the impact could be more serious.

Recommendation:

- Use parameterized SQL queries or prepared statements.
- Validate input appropriately on the server.
- Return generic error messages to clients.
- Log detailed diagnostic information securely on the server instead of returning it in HTTP responses.

Evidence: Burp Repeater request and response showing HTTP 500, SQLite syntax error and SQL query disclosure.

Status: Database error and information disclosure confirmed; SQL injection requires further validation.

Test 4 – Security Misconfiguration

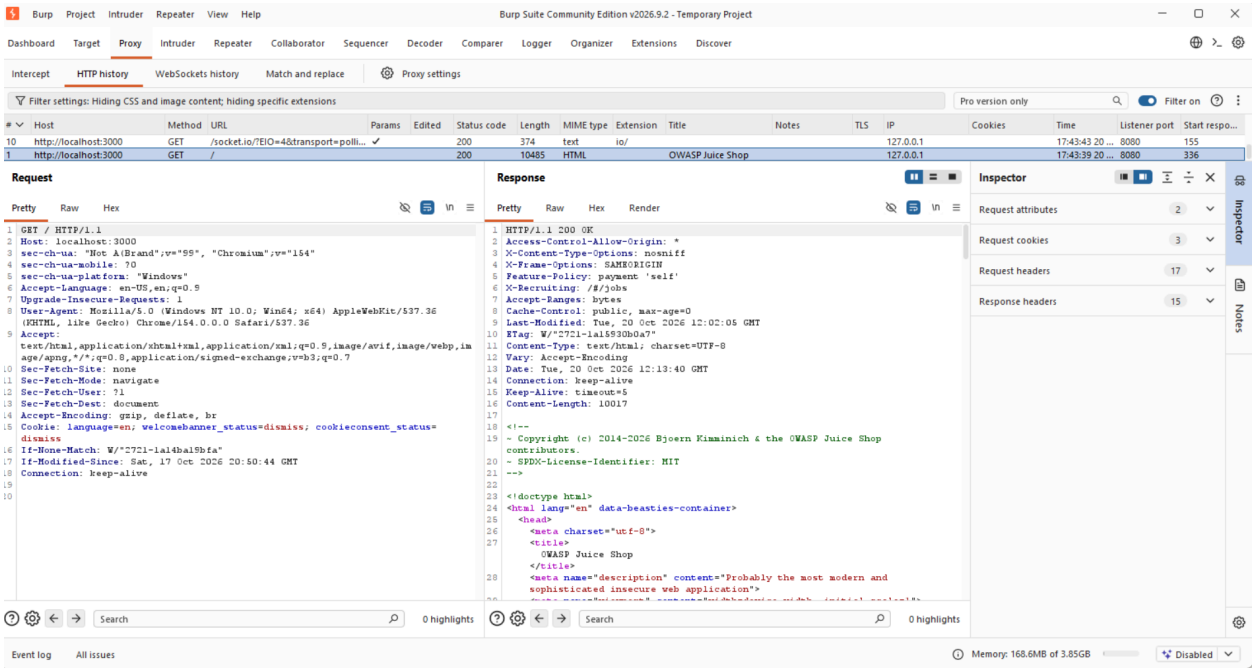
Procedure

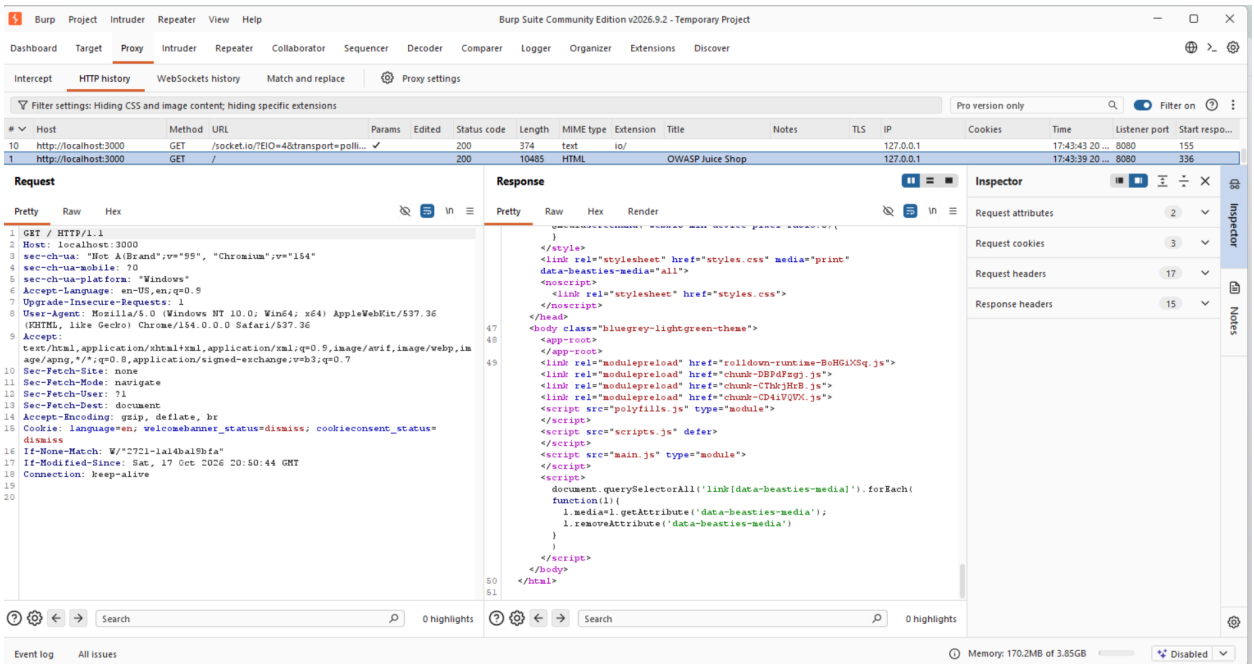
1. Inspect the response headers for the home page and an API request.
2. Look for headers such as Content-Security-Policy and X-Content-Type-Options.
3. Check error responses for unnecessary technical details or sensitive information.

Record: The missing or misconfigured control, its relevance, and any demonstrated impact. A missing header is not automatically a confirmed vulnerability.

Observations

The home page request (GET /) and its response were captured in Burp Suite HTTP history. The server returned status 200 OK.





- Security-related headers present in the response include X-Content-Type-Options: nosniff and X-Frame-Options: SAMEORIGIN.
- No Content-Security-Policy header appears in the captured response. On its own, this is not a confirmed vulnerability, and no direct impact was demonstrated.
- The response also contains Access-Control-Allow-Origin: *, Feature-Policy: payment 'self', and a custom X-Recruiting header. These are noted as configuration details, with no impact demonstrated.
- The error response observed in Test 3(c) is also relevant because it disclosed unnecessary technical details, including the database type, SQL query, and error information.

Result: No confirmed vulnerability was established from the captured home page response alone. The missing Content-Security-Policy header is recorded as a hardening observation only. The verbose database error from Test 3(c) is the clearer information-disclosure finding.

Test 5 – Cart and Business Logic

Procedure

1. Add an item to your cart normally.
2. Capture the cart-related request and send it to Repeater.
3. Inspect the quantity and price-related fields.

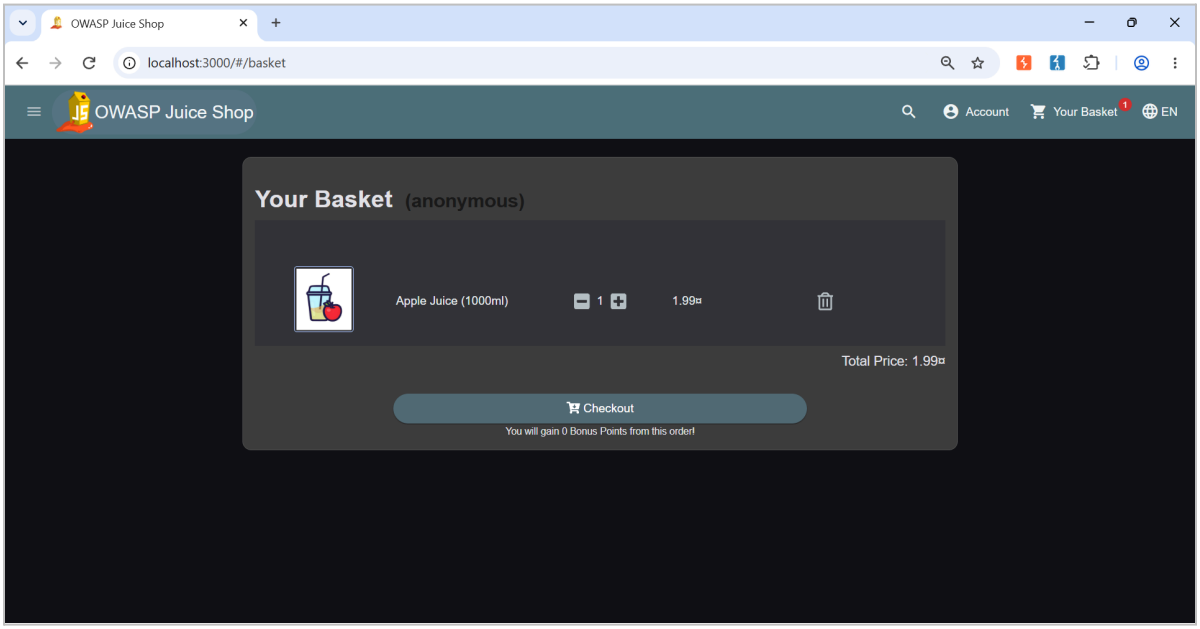
4. Test a harmless quantity change and check whether the server validates it and calculates the total correctly.

Record: Whether invalid values are rejected and whether prices and totals are enforced server-side. Test data only was used; no real payment services were connected.

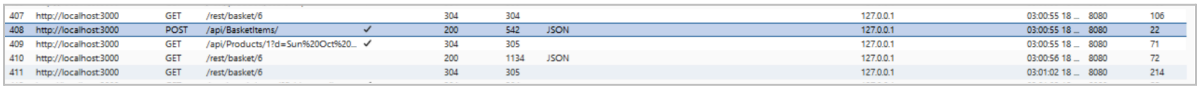
Observations

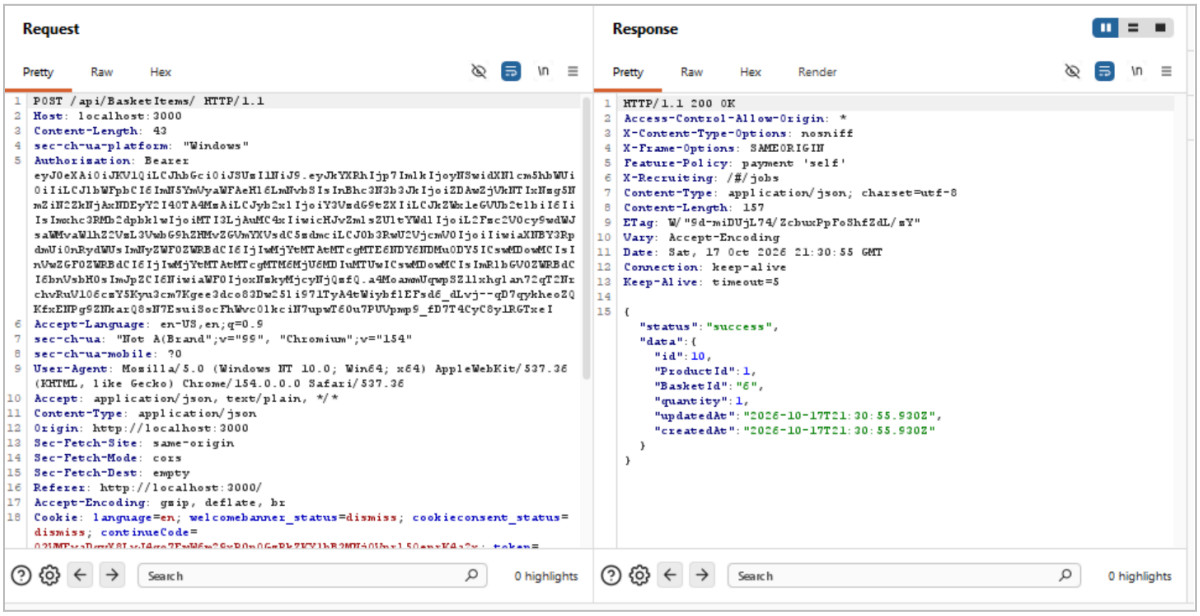
(a) Add to cart

A product was added to the basket, and the basket was opened to confirm that the product appears.



The request and response can be seen in Burp Suite.

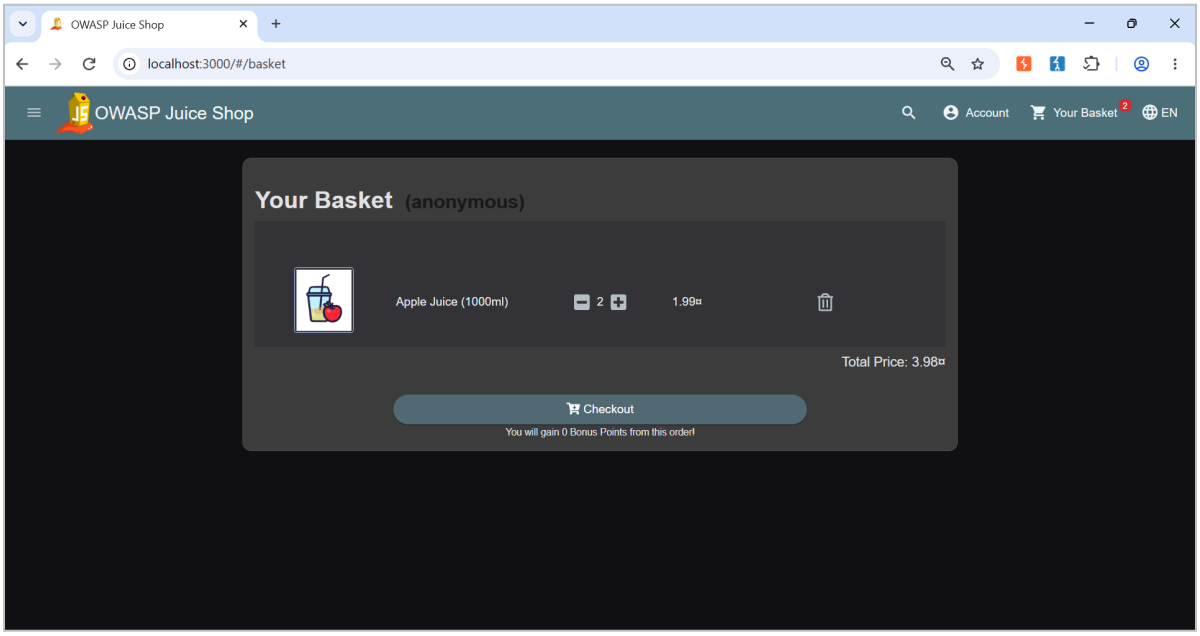


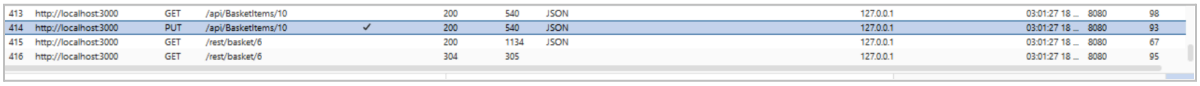


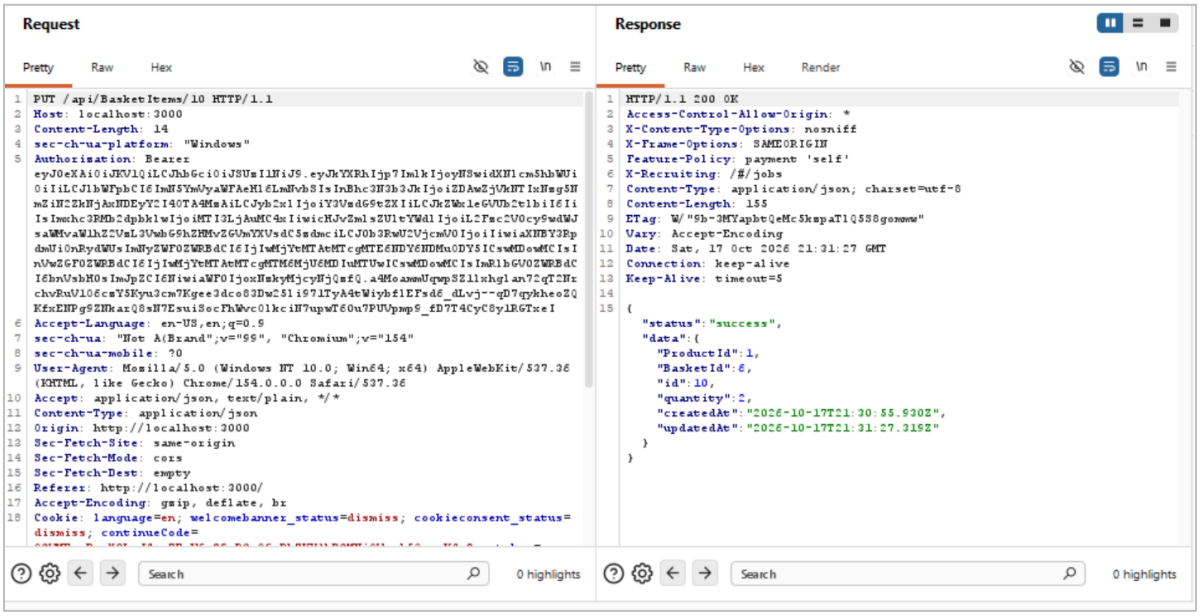
The response shows the correct quantity of 1.

(b) Quantity change

The quantity of the item was increased, and the request is visible in Burp Suite.





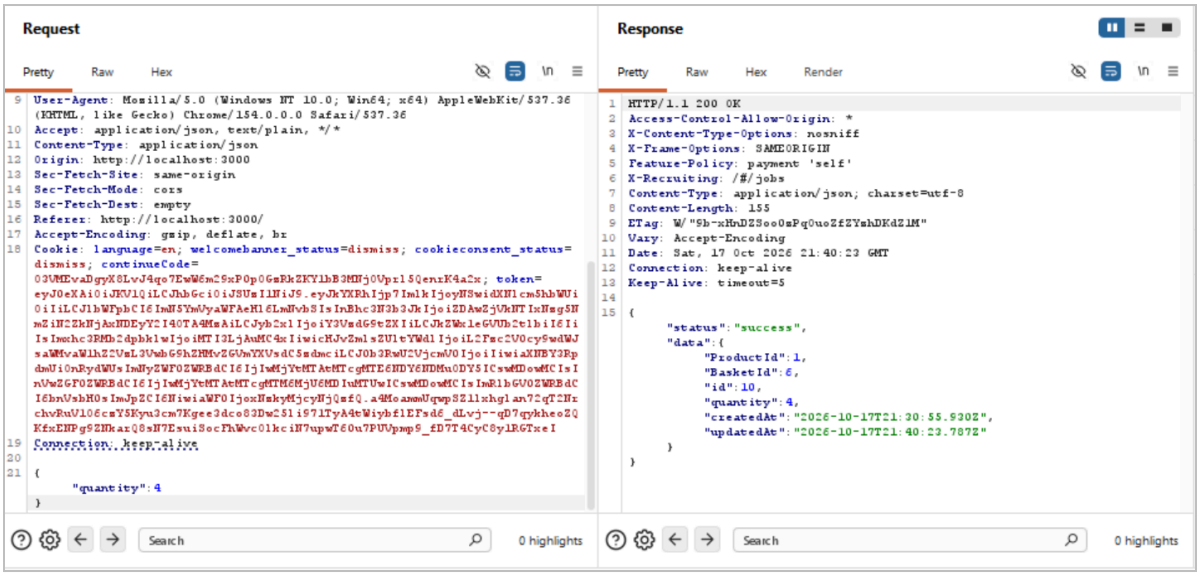


As seen in the response, the quantity is now 2.

(c) Input validation on quantity

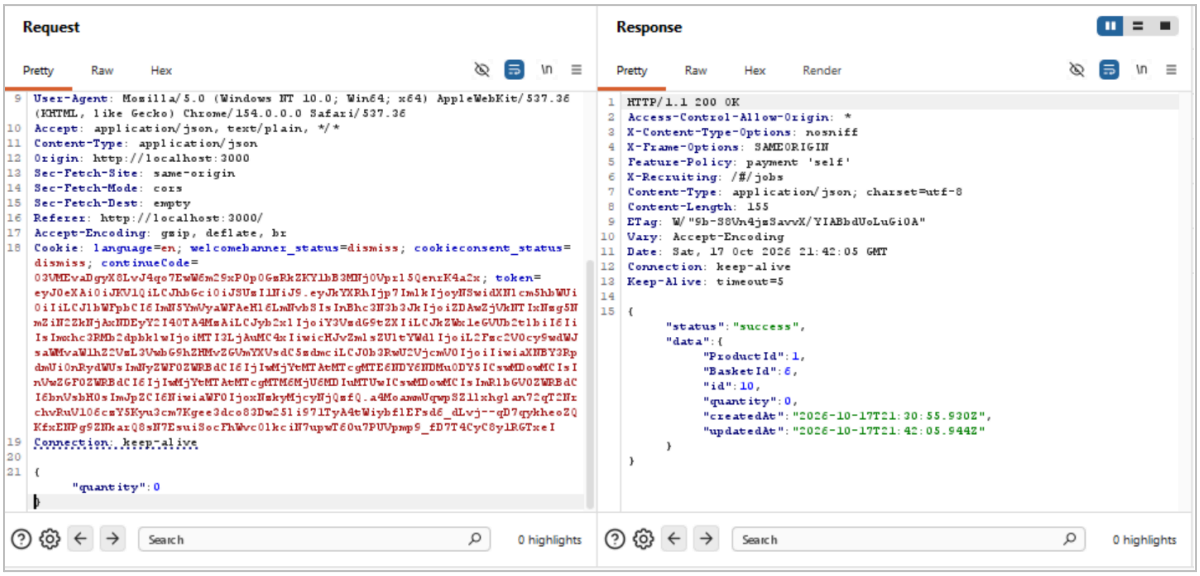
Test case	Input	Result
1	"quantity" = 4 (positive)	A normal positive quantity works.
2	"quantity" = 0	The server handles zero safely.
3	"quantity" = -1 (negative)	The server accepts -1. It should generally reject negative quantities.
4	"quantity" = abc (alphanumeric)	The server rejects the value, which is appropriate input validation.

Test case 1: positive quantity, "quantity" = 4.



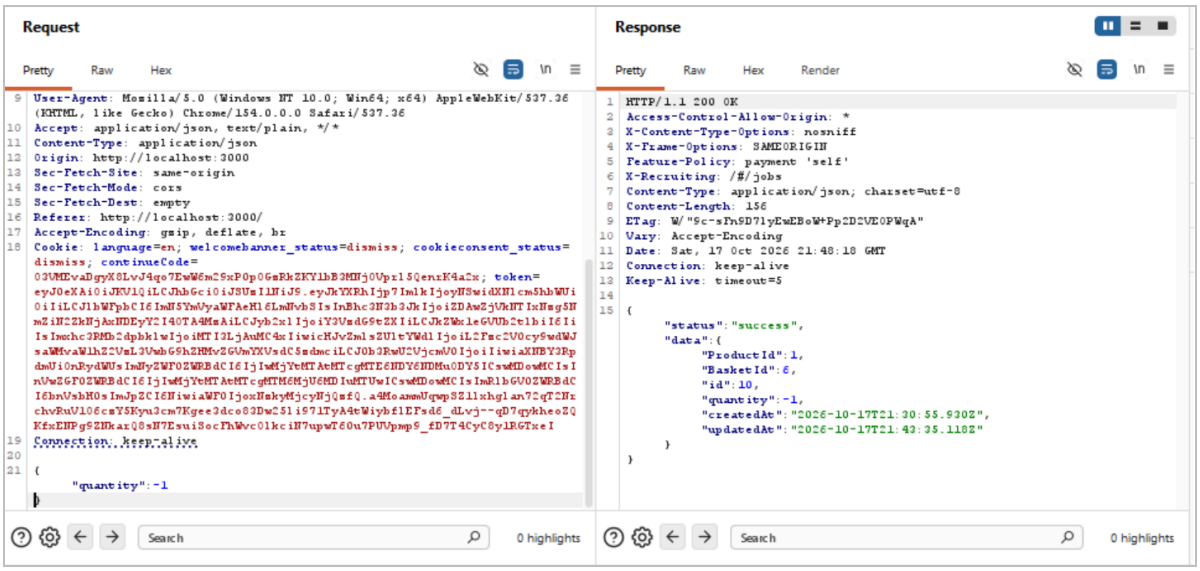
A normal positive quantity works.

Test case 2: "quantity" = 0.



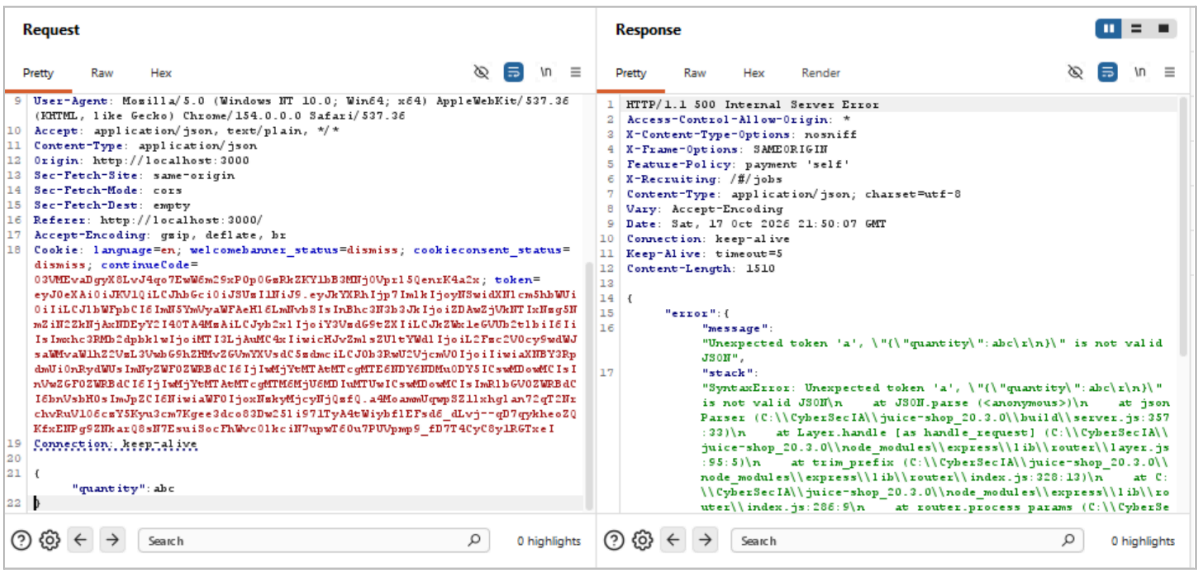
The server handles zero safely.

Test case 3: negative quantity, "quantity" = -1.



The server should generally reject negative quantities. Here, it accepts -1, which may cause an incorrect basket total, an invalid item or an unintended discount. If that impact is confirmed, it would demonstrate a business logic vulnerability.

Test case 4: alphanumeric value, "quantity" = abc.



Here the server rejects the input value. Rejecting abc is appropriate input validation, because a quantity should be numeric. Input validation works for this case.

Finding: Insufficient server-side validation of negative cart quantity.

Observation: During cart quantity testing, the application handled a quantity of zero safely and rejected an alphanumeric value. However, it did not reject a negative quantity (-1).

Impact: If negative quantities are accepted and affect basket contents or price calculations, an attacker may be able to manipulate cart behavior or totals.

Recommendation: Implement server-side validation to ensure quantities are numeric integers within an allowed range, typically greater than zero for basket items. Reject invalid values before updating the basket or calculating totals.

Severity: Potential business logic issue. Final severity depends on whether accepting a negative quantity produces an observable impact.

8. Analysis of Outcome

This section analyses the results of the five tests. The table below summarises the outcome of each test, followed by the main observations and the limits of the testing.

Test	Outcome	Status
1. Authentication and session management	Login succeeded with valid credentials and was rejected with an invalid password; logout returned an empty user object. No vulnerability shown.	Completed; no vulnerability shown
2. Broken access control	Account B and Account C each saw only their own profile data. No unauthorized access demonstrated.	Completed; no vulnerability shown
3. Input validation	Empty and long inputs handled safely. Unexpected punctuation caused HTTP 500 with SQLite error and SQL query disclosure; SQL injection needs further validation.	Completed; information disclosure confirmed
4. Security misconfiguration	Home page response had X-Content-Type-Options and X-Frame-Options but no Content-Security-Policy; no impact demonstrated.	Completed; hardening observation only
5. Cart and business logic	Zero handled safely and "abc" rejected, but a negative quantity (-1) was accepted.	Completed; potential business logic issue

- Burp Suite makes it easy to see exactly what the browser sends and what the server returns, which is essential for judging whether behaviour is secure.

- A suspicious result, such as an error message, a different response or a missing header, is only a lead. It becomes a finding only when its security impact is established.
- Even when injection is not yet proven, verbose error messages that expose database details are a real weakness because they help an attacker map the application.
- Server-side validation matters: the application rejected a non-numeric quantity but accepted a negative one, so every input needs checking against its allowed range.

9. Limitations of the testing

The results should be read within the limits of the manual testing carried out:

- Logout was checked by sending the request without a token. Replaying the old token after logout was not tested, so server-side invalidation of the session is unconfirmed.
- Broken access control was checked by comparing the profile pages of two accounts. A request for one account's resource using the other account's session was not attempted.
- The SQLite error in input validation is a lead for SQL injection. Injection itself was not confirmed.
- For the negative cart quantity, the effect on the basket total was not checked, so the severity remains potential.
- Only the home page response headers were reviewed for security misconfiguration.

10. Conclusion

This assessment used Burp Suite to study how a local OWASP Juice Shop instance handles registration, login, search and cart requests, and applied five security tests to it. Authentication behaved as expected, and no unauthorised access was shown in the comparison of two accounts. The main findings were the disclosure of database error details from unexpected search input and the acceptance of a negative cart quantity, which is a possible business logic flaw.

The results show why a server must validate every input, return generic error messages and enforce its checks on the server side. Further testing is needed to confirm SQL injection and the impact of the negative quantity, as noted under the limitations.

11. References

- OWASP Juice Shop project: https://owasp.org/www-project-juice-shop/
- PortSwigger, Burp Suite documentation: https://portswigger.net/burp/documentation
- OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing-guide/
- OWASP Top 10: https://owasp.org/Top10/